算法设计与分析

第四章 动态规划

Dynamic Programming

哈尔滨工业大学（深圳）
陈勇勇

D*ynamic Programming* is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems.

《算法导论》第15章

斐波那契数列

第1个月：有1对兔子(1)，没有繁殖能力，1+0共有1对兔子->　第1个月（1对）：

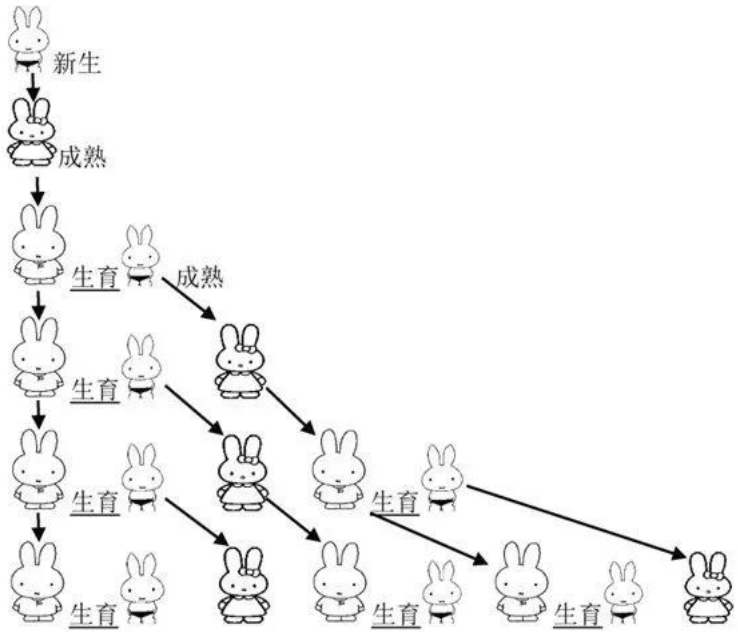
第2个月：兔子(1)进入成熟期，1+0共有1对兔子->　第2个月（1对）：

第3个月：兔子生了1对小兔子(2)，1+1共有2对兔子->　第3个月（2对）：

第4个月：(1)又生了兔子(3)，1+2共有3对兔子->　第4个月（3对）：

第5个月：(1)生了(4)，(2)生了(5)，2+3共有5对兔子->　第5个月（5对）：

第6个月：(1)(2)(3)各生下1对小兔子，3+5共有8对兔子->　第6个月（8对）：



……

每个月兔子数的数列是：

1, 1, 2, 3, 5, 8, 13, 21, 34, ...

因此：

$f(1)=1, f(2)=1,$

$f(3)=f(1)+f(2),$

$f(4)=f(3)+f(2), \cdots$

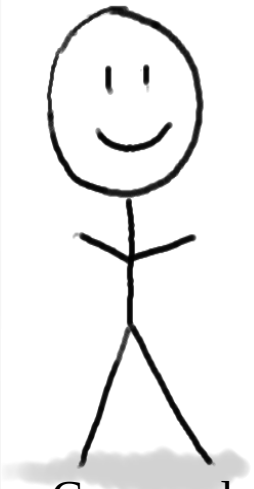
$$F(n)=\begin{cases}1 & ,n=1\\ 1 & ,n=2\\ F(n-1)+F(n-2) & ,n>2\end{cases}$$

斐波那契数列(Top-down)

$$F(n)=\begin{cases}1 & ,n=1\\ 1 & ,n=2\\ F(n-1)+F(n-2) & ,n>2\end{cases}$$

如何设计算法，编写程序，给定任意的n求$F(n)$

```
1. public int fib(int n) {
2.      if (n < 1) {
3.          return -1;
4.      }
5.      if (n == 1 || n == 2) {
6.          return 1;
7.     }
8.     return fib(n - 1) + fib(n - 2);
9.   }
```

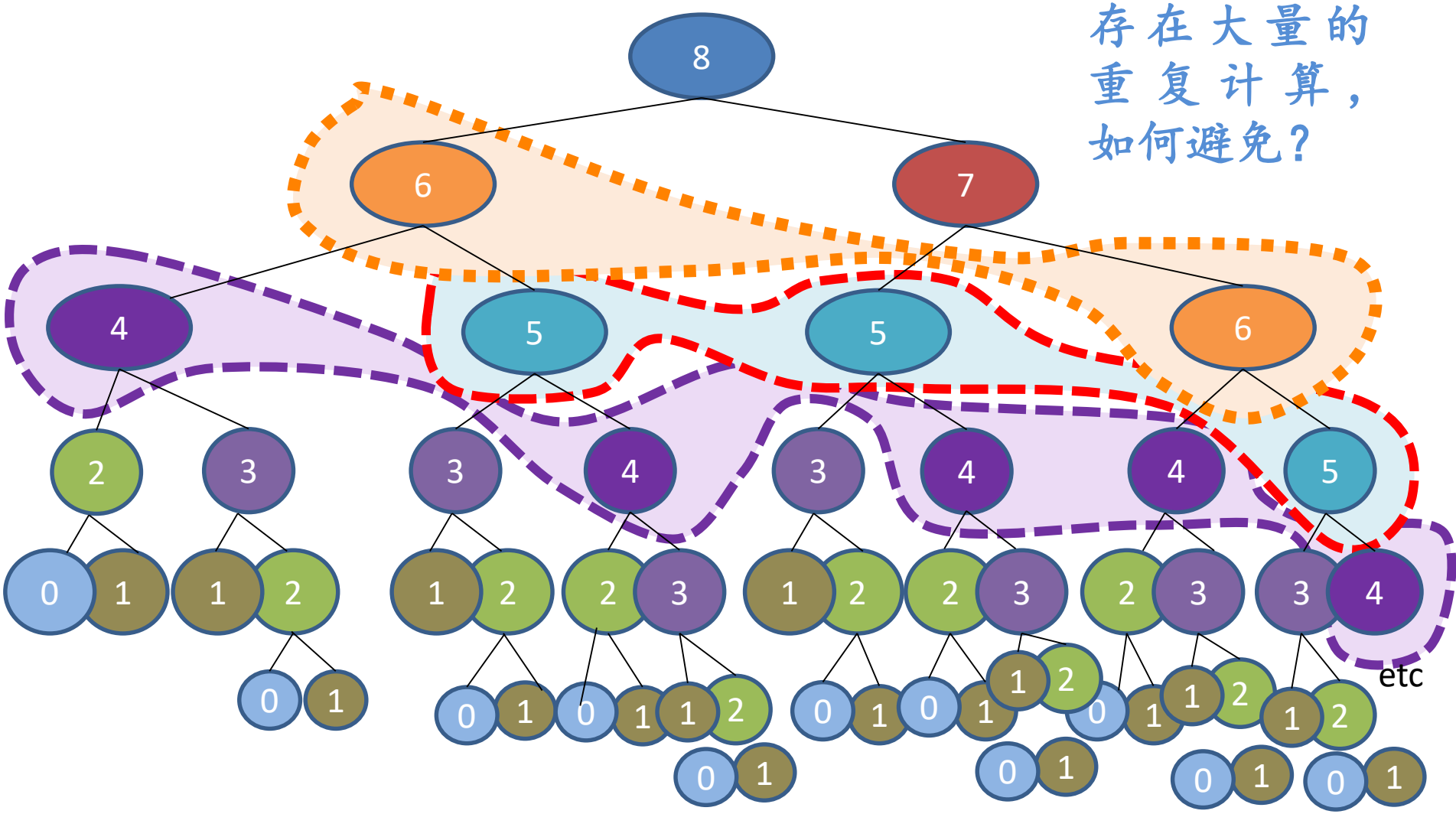


Can we do better ?

时间复杂度：$O(2^n)$

斐波那契数列

存在大量的重复计算，如何避免？



本讲内容

4.1 动态规划的原理

4.2 矩阵乘法问题

4.3 最长公共子序列

4.4 0-1背包问题

4.5 最优二叉搜索树

动态规划 = 分治递归 + 记忆存储

- 动态规划是一种算法设计的范式（paradigm）或思想
  不是解决某一具体问题的具体算法

- 理查德·贝尔曼（Richard Bellman）在1950年代首次提出了这个名字，当时他正为美国兰德公司工作，主要为美国空军和政府项目服务。

> “It’s impossible to use the word, dynamic, in the pejorative sense … I thought dynamic programming was a good name. It was something not even a Congressman could object to.”
>
> ——*理查德·贝尔曼*



理查德·贝尔曼（1920年8月26日－1984年3月19日），美国应用数学家，美国国家科学院院士，动态规划的创始人。

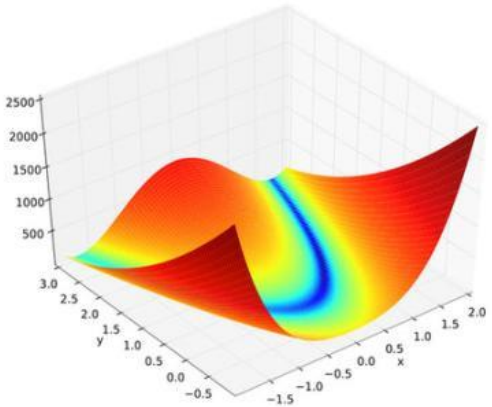
Why?

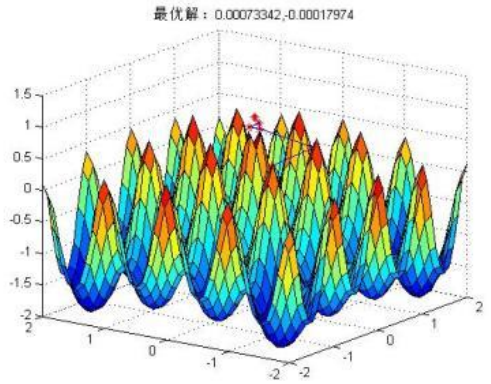
“那些遗忘过去的人注定要重蹈覆辙。”——乔治·桑塔亚纳

- 分治技术（无记忆递归）
  - 子问题是相互独立的
  - 如果子问题不是相互独立的，分治方法将重复计算公共子问题，效率很低

What?

- 动态规划的特点（有记忆迭代）
  - 把原始问题划分成一系列子问题
  - 求解每个子问题仅一次，并将其结果保存在一个表中，以后用到时直接存取，不重复计算，节省计算时间
  - 自底向上地计算
- 适用范围（优化问题）
  - 优化问题给定一组约束条件和一个代价函数，在解空间中搜索具有最小或最大代价的优化解
  - 很多优化问题可分为多个子问题，子问题相互关联，子问题的解被重复使用





How?

- 使用动态规划的条件
  - 优化子结构(正确性)
    - 当一个问题的优化解包含了子问题的优化解时，我们称这个问题具有优化子结构。
    - 缩小子问题集合，只需那些优化问题中包含的子问题，降低实现复杂性
    - 优化子结构使得我们能自下而上地完成求解过程
  - 重叠子问题(有效性)
    - 在问题的求解过程中，很多子问题的解将被多次使用

- 动态规划算法的设计步骤

  ① 分析优化解的结构

  ② 递归地定义最优值的代价

  ③ 自底向上地计算优化解的代价保存之，并获取构造最优值的信息

  ④ 根据构造最优值的信息构造优化解

- 步骤①–③是动态规划的基本步骤。在只需要计算出最优值的情形，步骤④将被省略。
- 若求出原问题的一个优化解，则必须执行步骤④。需要通过步骤③中记录信息得到优化解。

本讲内容

4.1 动态规划的原理

4.2 矩阵乘法问题

4.3 最长公共子序列

4.4 0-1背包问题

4.5 最优二叉搜索树

问题的定义

- 输入：$< A_1, A_2, \ldots, A_n >$， $A_i \in \mathbb{R}^{p_{i-1} \times p_i}$是矩阵
- 输出：计算$A_1 \times A_2 \times \cdots \times A_n$的最小代价方法

矩阵乘法的代价/复杂性： 乘法的次数

若A是$p \times q$矩阵， B是$q \times r$矩阵，则$A \times B$的代价是$O(pqr)$

动机

- 矩阵链乘法的实现
  - 矩阵乘法满足结合率。
  - 计算一个矩阵链的乘法可有多种方法：

例如，

$$
\begin{aligned}
& A_1 \times A_2 \times A_3 \times A_4 \\
&= \left(A_1 \times \left(A_2 \times (A_3 \times A_4)\right)\right) \\
&= \left((A_1 \times A_2) \times (A_3 \times A_4)\right) \\
&= \left(\left((A_1 \times A_2) \times A_3\right) \times A_4\right) \\
&= \left(A_1 \times \left((A_2 \times A_3) \times A_4\right)\right) \\
&= \left(\left(A_1 \times (A_2 \times A_3)\right) \times A_4\right)
\end{aligned}
$$

- 矩阵链乘法的代价与计算顺序的关系

  – 设 $A_1 = 10 \times 100$ 矩阵， $A_2 = 100 \times 5$ 矩阵，

  $A_3 = 5 \times 50$ 矩阵

$$T\big((A_1 \times A_2) \times A_3\big) = 10 \times 100 \times 5 + 10 \times 5 \times 50$$

$$= 7500$$

$$T(A_1 \times (A_2 \times A_3)) = 100 \times 5 \times 50 + 10 \times 100 \times 50$$

$$= 75000$$

结论：不同计算顺序有不同的代价

- 矩阵链乘法优化问题的解空间(穷举法)
  - 设$p(n)$=计算n个矩阵乘积的方法数
  - $p(n)$的递归方程

$$(A_1 \times \cdots \times A_k) \times (A_{k+1} \times \cdots \times A_n)$$

$$p(n) = \begin{cases} 1 & ,n = 1 \\ \sum_{k=1}^{n-1} p(k)p(n-k), & n > 1 \end{cases}$$

$$p(n) = C(n-1) = Catalan\text{数} = \frac{1}{n}\binom{2(n-1)}{n-1} = \frac{(2(n-1))!}{n!(n-1)!}$$

$$= \Omega(4^n/n^{3/2})$$

如此之大的解空间是无法用枚举方法求出最优解的！

矩阵链乘法问题的动态规划算法

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

第一步：分析优化解的结构

- 两个记号
  - $A_{i\sim j} = A_i \times A_{i+1} \times \cdots \times A_j$
  - $\mathbf{cost}(A_{i\sim j})$ =计算$A_{i\sim j}$的代价

第一步：分析优化解的结构

- **优化解的结构**（任何最优解都是由子问题的最优解构成）
  - 若计算$A_{1\sim n}$的优化顺序在k处断开矩阵链，即$A^1_{1\sim n}=A^1_{1\sim k}\times A^1_{k+1\sim n}$，则在$A^1_{1\sim n}$的优化顺序中，对应于子问题$A^1_{1\sim k}$的解必定是$A_{1\sim k}$的优化解；
  - 对应于子问题$A^1_{k+1\sim n}$的解必须是$A_{k+1\sim n}$的优化解。

证明：（反证法）

假设最优矩阵链乘$A^1_{1\sim n}=(A_1\times\cdots\times A_k)\times(A_{k+1}\times\cdots\times A_n)$

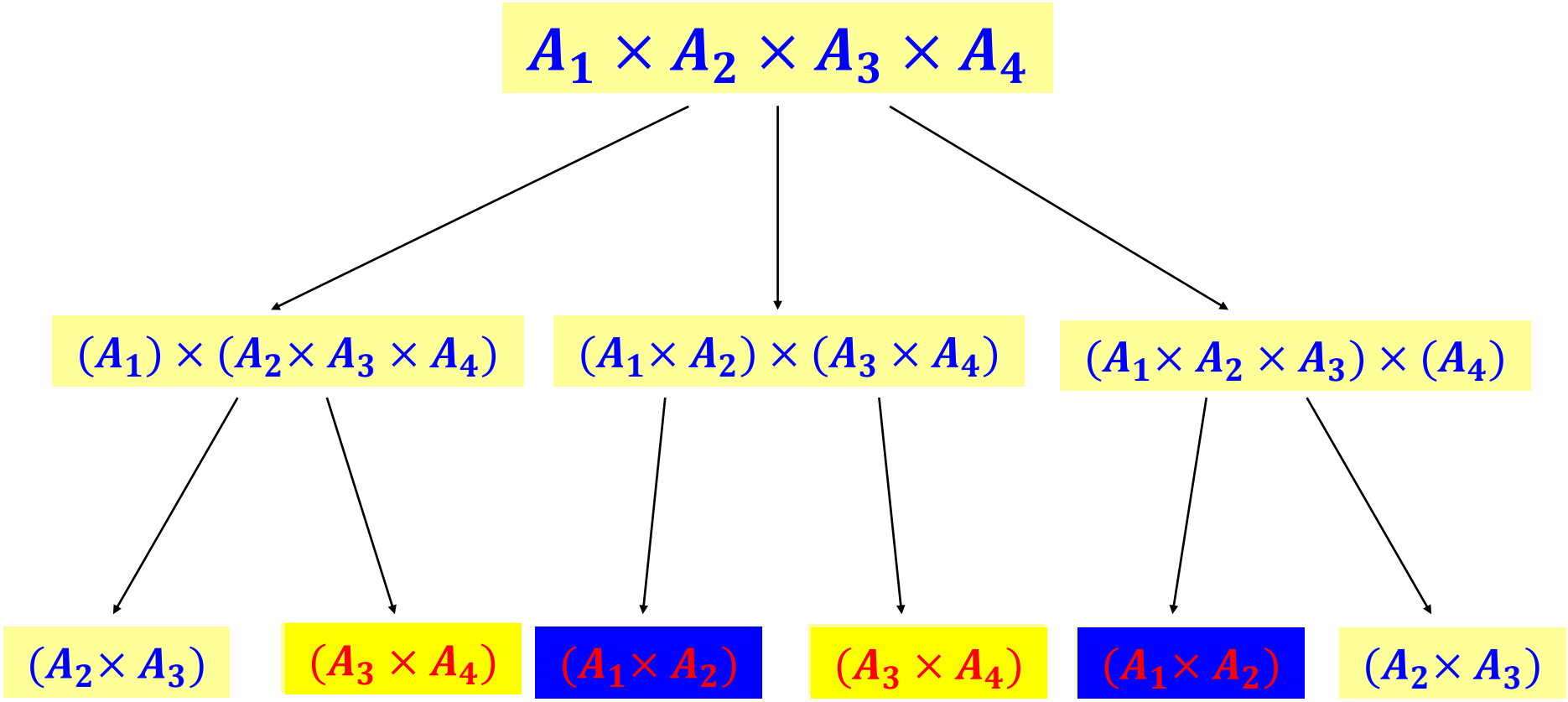
假设$A^1_{1\sim k}=A_1\times(A_2\times A_3)\times\cdots\times A_k$

$A^1_{k+1\sim n}=A_{k+1}\times\cdots\times(A_{n-1}\times A_n)$

反证法：假设有一个$A^2_{1\sim k}=A_1\times\cdots\times(A_{k-1}\times A_k)$比$A^1_{1\sim k}$更优，那么$A^2_{1\sim n}=A^2_{1\sim k}\times A^1_{k+1\sim n}$将构成$A_{1\sim n}$的一个新矩阵链乘。显然$A^2_{1\sim n}$的计算代价比$A^1_{1\sim n}$少，这与$A^1_{1\sim n}$是$A_{1\sim n}$的最优矩阵链乘相矛盾。

具有优化子结构：问题的优化解包括子问题优化解

- 子问题重叠性



具有子问题重叠性

第二步：递归地定义最优值的代价

- 假设
  - $m[i,j]$ = 计算$A_{i\sim j}$的最小乘法数
  - $m[1,n]$ = 计算$A_{1\sim n}$的最小乘法数
  - $A_1 \cdots A_k A_{k+1} \cdots A_n$是优化解（$k$实际上是不可预知）
- 代价方程

$$m[i,i] = \text{计算} A_{i\sim i} \text{的最小乘法数} = 0$$

$$m[i,j] = min_{i\le k<j}\{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\}$$

其中，$p_{i-1}p_k p_j$是计算$A_{i\sim k} \times A_{k+1\sim j}$所需乘法数，$A_{i\sim k}$和$A_{k+1\sim j}$分别是$p_{i-1} \times p_k$和$p_k \times p_j$矩阵。

考虑到所有的k，优化解的代价方程为

$$m[i,j]=\begin{cases}0 & if\ \ i=j\\ min_{i\le k<j}\{m[i,k]+m[k+1,j]+p_{i-1}p_kp_j\} & if\ \ i<j\end{cases}$$

代价矩阵m的维度是？

代价矩阵m是上三角？

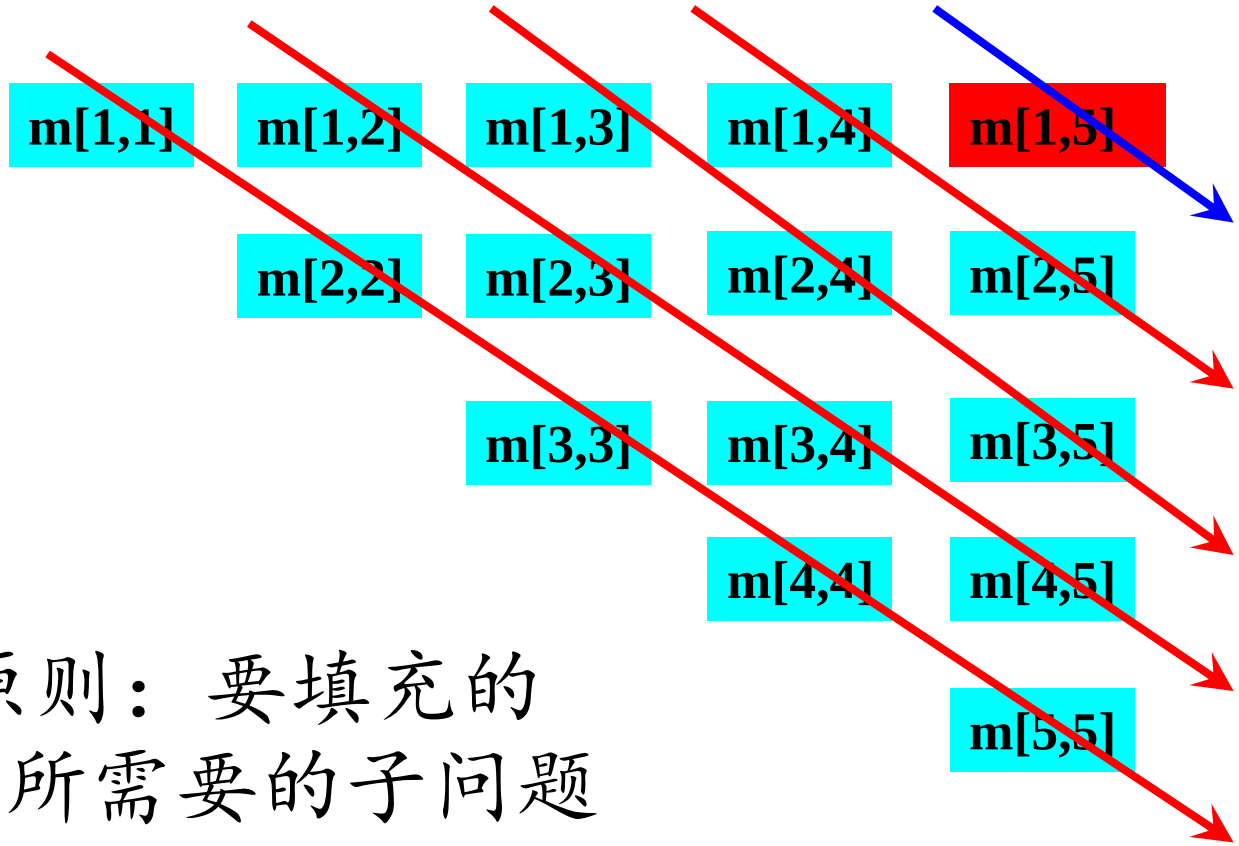
第三步：自底向上计算优化解的代价

$$m[i,j] = min_{i \le k < j}\{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\}$$

m[1,1]	m[1,2]	m[1,3]	m[1,4]	m[1,5]
	m[2,2]	m[2,3]	m[2,4]	m[2,5]
		m[3,3]	m[3,4]	m[3,5]
			m[4,4]	m[4,5]
				m[5,5]

$$m[1,5] = min\begin{cases} m[1,1] + m[2,5] + p_0 p_1 p_5 & k = 1 \\ m[1,2] + m[3,5] + p_0 p_2 p_5 & k = 2 \\ m[1,3] + m[4,5] + p_0 p_3 p_5 & k = 3 \\ m[1,4] + m[5,5] + p_0 p_4 p_5 & k = 4 \end{cases}$$

$$m[i,j] = min_{i \le k < j}\{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\}$$



填充原则：要填充的$m[i,j]$所需要的子问题的代价都是已知的

算法

Matrix-Chain-Order

Input: 向量p, p中存放各个矩阵的行数列数

Output: m 完成矩阵链乘的最小乘法数

过程：Matrix-Chain-Order(p)

```
1.  n←length(p)-1;
2. FOR   i←1   TO   n   DO
3.          m[i, i]←0;      /*初始化主对角线*/
4. FOR   l←2   TO   n DO    /*计算第l个对角线*/
5.     FOR   i←1   TO   n-l+1   DO
6.          j ← i+l-1;
7.          m[i, j] ← ∞;
8.          FOR   k ← i   TO   j-1   DO    /* 计算m[i,j] */
9.              q ← m[i, k] + m[k+1, j] + p_{i-1} p_k p_j;
10.             IF   q<m[i, j]   THEN   m[i,j] ← q;
11. Return m.
```

考虑p的维度是？

第四步：获取构造最优值的信息

Matrix-Chain-Order

Input: 向量p, p中存放各个矩阵的行数列数

Output: m 完成矩阵链乘的最小乘法数

过程：Matrix-Chain-Order(p)

$S[i,j]$记录$A_iA_{i+1}\cdots A_j$的最优划分处

```
1.  n ← length(p)-1;
2. FOR   i ← 1   TO   n   DO
3.       m[i, i] ← 0, S[i, i] ← 0;      /*初始化主对角线*/
4. FOR   l ← 2   TO   n DO    /*计算第l对角线 */
5.    FOR   i ← 1   TO   n-l+1   DO
6.        j ← i+l-1;
7.        m[i, j] ← ∞;
8.        FOR   k ← i   TO   j-1   DO    /* 计算m[i,j] */
9.            q ← m[i, k] + m[k+1, j] + p_{i-1} p_k p_j;
10.           IF   q<m[i, j]   THEN   m[i,j] ← q, S[i,j] = k;
11. Return m,S.
```

构造最优解

Print-Optimal-Parens(S, i, j)

```
1. IF j = i
2. THEN  Print "A_i";
3. ELSE   Print "("
4.      Print-Optimal-Parens(S, i, S[i, j])
5.      Print-Optimal-Parens(S, S[i, j]+1, j)
6.      Print  ")"
```

$S[i,j]$ 记录$A_i \cdots A_j$的最优划分处；
$S[i,S[i,j]]$记录$A_i \cdots A_{S[i,j]}$ 的最优划分处；
$S[S[i,j]+1,j]$ 记录$A_{S[i,j]+1} \cdots A_j$的最优划分处。

调用Print-Optimal-Parens$(S,1,n)$
即可输出$A_{1\sim n}$的优化计算顺序

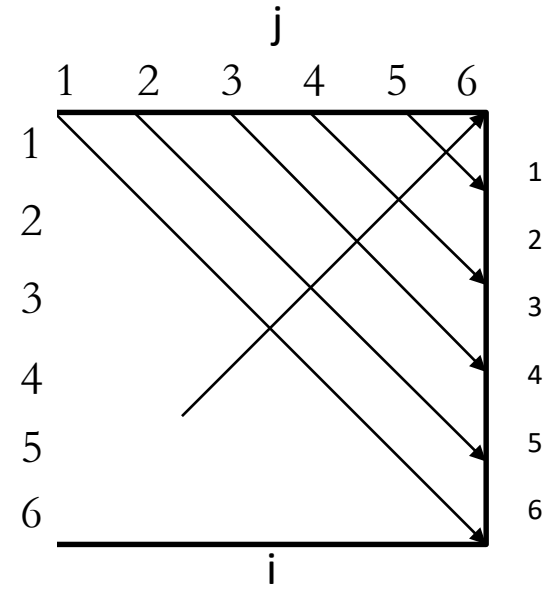
算法复杂性

- 时间复杂性
  - 计算代价的时间
    - (l, i, k)三层循环，每层至多$n-1$步
    - $O(n^3)$
  - 构造最优解的时间：$O(n)$
  - 总时间复杂性为：$O(n^3)$
- 空间复杂性
  - 使用数组p, m和S
  - 需要空间$O(n^2)$

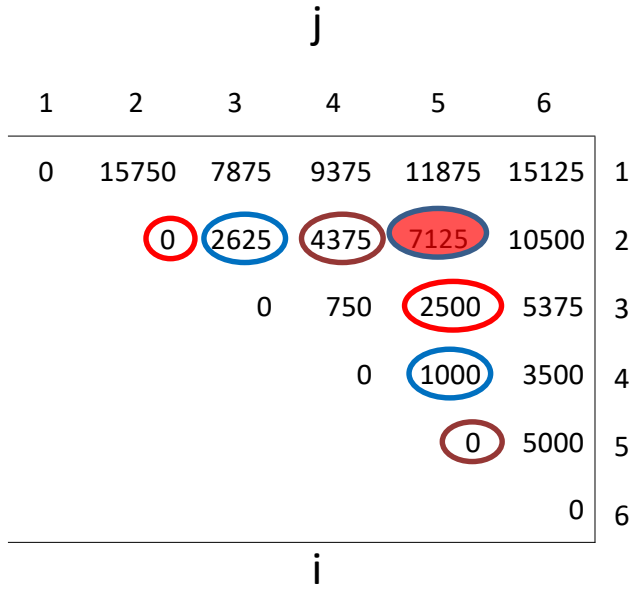
示例

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

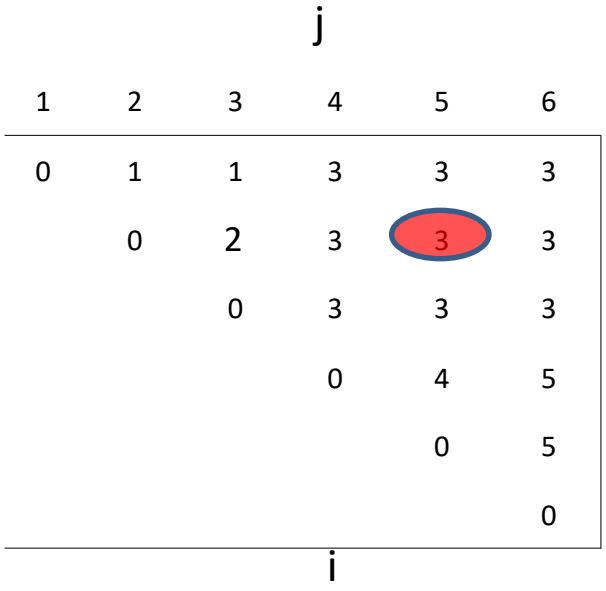
$$m[i,j] = min_{i \le k < j}\{m[i,k] + m[k+1,j] + p_{i-1}p_k p_j\}$$



（a）计算次序



（b）$m[i,j]$



（c）$S[i,j]$

$$m[2,5] = min\begin{cases} m[2,2] + m[3,5] + p_1p_2p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 & \boldsymbol{k=2} \\ m[2,3] + m[4,5] + p_1p_3p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 & \boldsymbol{k=3} \\ m[2,4] + m[5,5] + p_1p_4p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 & \boldsymbol{k=4} \end{cases}$$

$$\left(A_1 \times (A_2 \times A_3)\right) \times \left((A_4 \times A_5) \times A_6\right)$$

本讲内容

4.1 动态规划的原理

4.2 矩阵乘法问题

4.3 最长公共子序列

4.4 0-1背包问题

4.5 最优二叉搜索树

这两个物种有多相似



DNA:

AGCCCTAAGGGCTACCTAGCTT



DNA:

GACAGCCTACAAGCGTTAGCTTG

最长公共子序列



DNA:

AGCCCTAAGGGCTACCTAGCTT



DNA:

GACAGCCTACAAGCGTTAGCTTG

AGCCTAAGCTTAGCTT

最长公共子序列 问题的定义

- 子序列（Subsequence）
  - $X = (A, B, C, B, D, B)$
  - $Z = (B, C, D, B)$是X的子序列
  - $W = (B, D, A)$不是X的子序列
- 公共子序列（Common Subsequence）
  - Z是序列X与Y的公共子序列，如果Z是X的子序也是Y的子序列。

问题的定义

最长公共子序列（LCS）问题：

输入：$X=(x_1,x_2,\cdots,x_n)$

$Y=(y_1,y_2,\cdots,y_m)$

输出：Z = X与Y的最长公共子序列

- 利用暴力搜索方法求解LCS问题，即穷举X所有的子序列，检查每个子序列是否为Y的子序列。
- 序列X的每个子序列的下标集合为$\{1,2,\cdots,n\}$的子集，所以共有2^n个子序列。
- 暴力搜索方法的时间复杂度是指数阶的。

最长公共子序列结构分析

- 第i前缀
  - 设$X=(x_1,x_2,\cdots,x_n)$是一个序列，X的第i前缀X_i是一个序列，定义为$X_i=(x_1,x_2,\cdots,x_i)$

例. $X=(A,B,D,C,A)$，$X_1=(A)$，$X_2=(A,B)$，$X_3=(A,B,D)$

使用动态规划

- 分析优化解的结构 
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

第一步：分析优化解的结构

定理（优化子结构）： 设$X=(x_1,\cdots,x_m)$、$Y=(y_1,\cdots,y_n)$是两个序列，$Z=(z_1,\cdots,z_k)$是X与Y的LCS，我们有：

（1）如果$x_m=y_n$，则$z_k=x_m=y_n$，Z_{k-1}是X_{m-1}和Y_{n-1}的LCS，即，$LCS_{XY}=LCS_{X_{m-1}Y_{n-1}}+<x_m=y_n>$；

（2）如果$x_m\neq y_n$，且$z_k\neq x_m$，则Z是X_{m-1}和Y的LCS，即$LCS_{XY}=LCS_{X_{m-1}Y}$；

（3）如果$x_m\neq y_n$，且$z_k\neq y_n$，则Z是X与Y_{n-1}的LCS，即$LCS_{XY}=LCS_{XY_{n-1}}$。

（1） $X=(x_1,\cdots,x_{m-1},x_m)$，$Y=(y_1,\cdots,y_{n-1},x_m)$，$Z=(z_1,\cdots,z_k)$ 是X与Y的LCS，则

$$LCS_{XY}=LCS_{X_{m-1}Y_{n-1}}+<x_m=y_n>$$

证明：（反证法）

- 设$z_k\neq x_m$，则可加$x_m=y_n$追加到Z的末尾，得到一个长为$k+1$的X与Y的公共序列，与Z是X和Y的LCS矛盾，于是$z_k=x_m=y_n$。
- 现在证明Z_{k-1}是X_{m-1}与Y_{n-1}的LCS。显然Z_{k-1}是X_{m-1}与Y_{n-1}的公共序列。我们需要证明Z_{k-1}是LCS。设不然，则存在X_{m-1}与Y_{n-1}的公共子序列W，W的长大于$k-1$。增加$x_m=y_n$到W，我们得到一个长大于k的X与Y的公共序列，与Z是X和Y的LCS矛盾。于是Z_{k-1}是X_{m-1}与Y_{n-1}的LCS。

（2）$X=(x_1,\cdots,x_{m-1},x_m)$，$Y=(y_1,\cdots,y_{n-1},y_n)$，

$x_m \neq y_n$，且$z_k \neq x_m$，则$LCS_{XY}=LCS_{X_{m-1}Y}$

证明：（反证法）

由于$z_k \neq x_m$，Z是X_{m-1}与Y的公共子序列。我们来证Z是X_{m-1}与Y的LCS。设X_{m-1}与Y有一个公共子序列W，W的长度大于k，则W也是X与Y的公共子序列，与Z是LCS矛盾。

（3） 同（2）可证。

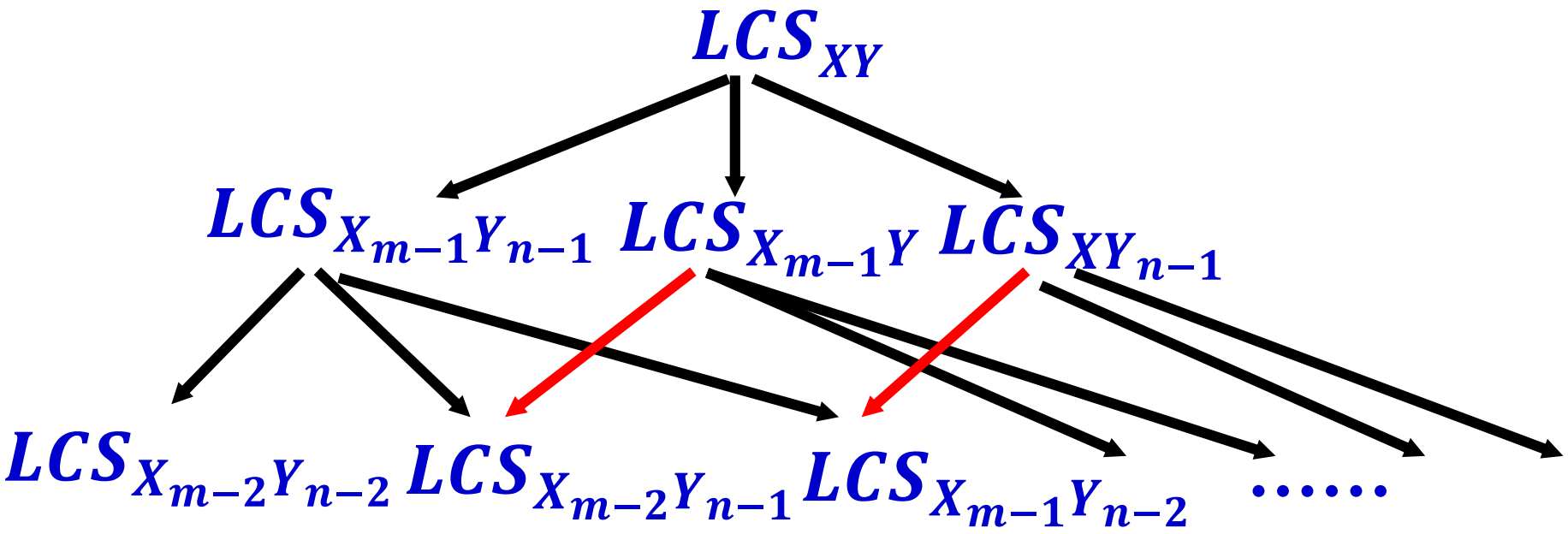
定理1表明：两个序列的LCS包含两个序列前缀的LCS。

因此，LCS问题具有优化子结构。可以考虑动态规划。

X和Y的LCS的优化解结构为

$$LCS_{XY}=\begin{cases} LCS_{X_{m-1}Y_{n-1}}+<x_m=y_n> & if\ \ x_m=y_n \\ LCS_{X_{m-1}Y} & if\ \ x_m\neq y_n, z_k\neq x_m \\ LCS_{XY_{n-1}} & if\ \ x_m\neq y_n, z_k\neq y_n \end{cases}$$

- 子问题重叠性



LCS问题具有子问题重叠性

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价 
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

第二步：递归地定义最优值的代价

建立LCS长度的递归方程

- $C[i,j]=$ 前缀X_i与前缀Y_j的LCS的长度
- LCS长度的递归方程

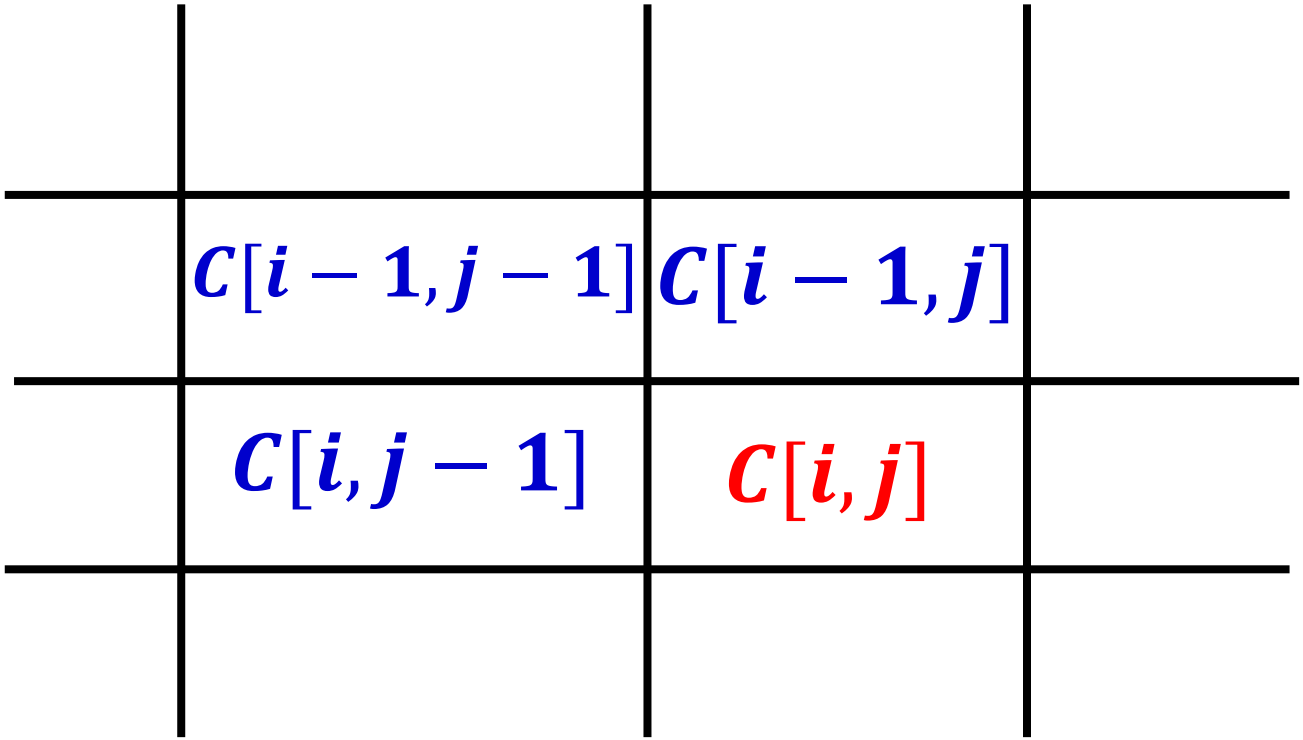
$$C[i,j]=\begin{cases}0 & if\ \ i=0\ \ 或\ \ j=0\\ C[i-1,j-1]+1 & if\ i,j>0\ 且\ x_i=y_j\\ Max(C[i,j-1],C[i-1,j]) & if\ \ i,j>0\ 且\ x_i\neq y_j\end{cases}$$

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息 
- 根据构造最优值的信息构造优化解

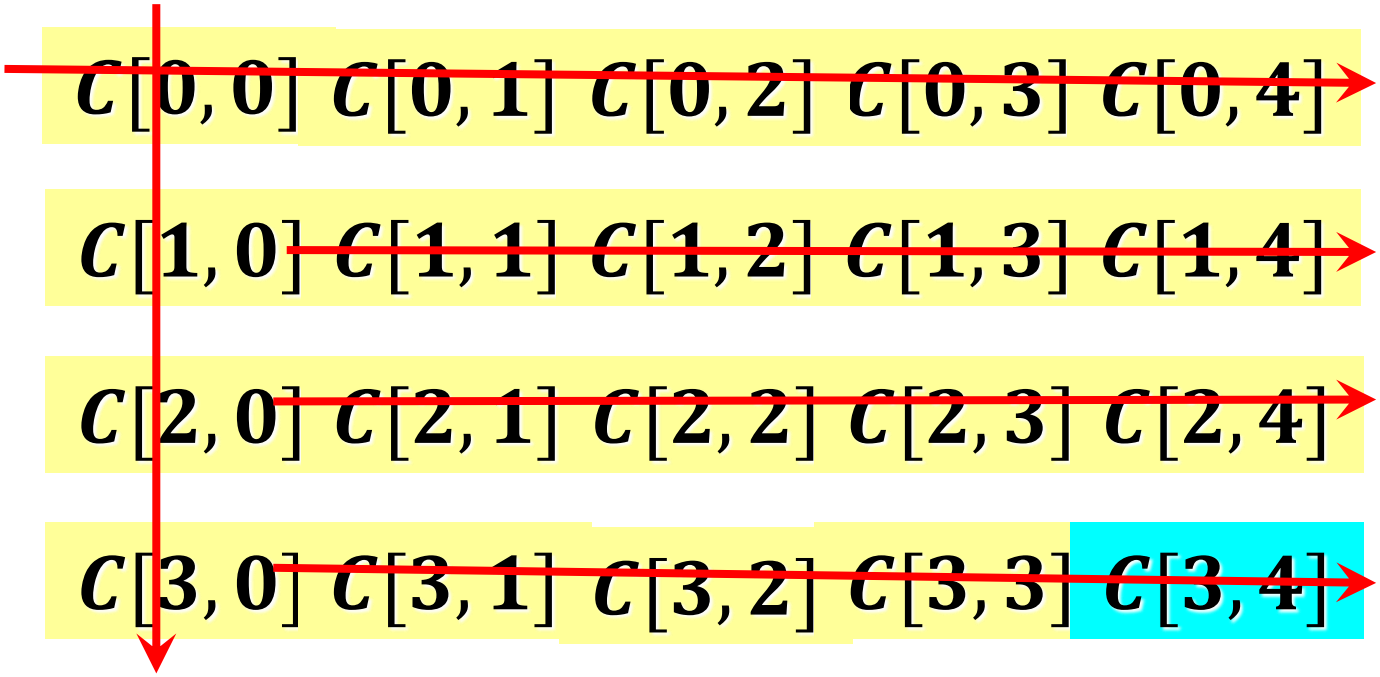
第三步：自底向上计算LCS的长度

- 基本思想



$$C[i,j]=\begin{cases}0 & if\ \ i=0\ \ 或\ \ j=0\\ C[i-1,j-1]+1 & if\ i,j>0\ 且\ x_i=y_i\\ Max(C[i,j-1],C[i-1,j]) & if\ \ i,j>0\ 且\ x_i\neq y_i\end{cases}$$

- 计算过程



填充原则：要填充的$C[i,j]$所需要的子问题的代价都是已知的

- 计算LCS长度的算法

  – 数据结构

  $C[0:m,0:n]$: $C[i,j]$是X_i与Y_j的LCS的长度

  $B[1:m,1:n]$: $B[i,j]$是指针，指向计算$C[i,j]$时所选择的子问题的优化解所对应的C表的表项

算法

LCS-length(X,Y)

Input: 序列X和Y

Output: C最长公共子序列长度，B构造优化解信息

过程：**LCS-length(X, Y)**

```
1. m ← length(X); n ← length(Y);
2. For   i ← 1   To   m   Do   C[i,0] ← 0;  /*初始化第0列*/
3. For   j ← 1   To   n   Do   C[0,j] ← 0;  /*初始化第0行*/
4. For   i ← 1   To   m   Do
5.     For   j ← 1   To   n   Do
6.         IF   x_i = y_j
7.           Then C[i,j] ← C[i-1,j-1]+1; B[i,j] ←"↖";
8.         Else IF C[i-1,j] ≥ C[i,j-1]
9.           Then C[i,j] ← C[i-1,j]; B[i,j] ←"↑";
10.         Else C[i,j] ← C[i,j-1]; B[i,j] ←"←";
11.Return C and B
```

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解 

第四步：获取构造最优值的信息

- 基本思想
  - 从$B[m,n]$开始按指针搜索
  - 若$B[i,j]=$“↖”，则$x_i=y_j$是LCS的一个元素
  - 如此找到的“LCS”是X与Y的LCS

构造最优解

```
Print-LCS(B,X,i,j)
1.IF  i=0  or  j=0  THEN  Return;
2.IF  B[i, j]="↖"
3.    THEN  Print-LCS(B, X, i-1, j-1), Print x_i
4.    ELSE    IF    B[i, j] = "↑"
5.         THEN   Print-LCS(B, X, i-1, j)
6.         ELSE   Print-LCS(B, X, i, j-1)
```

$Print-LCS(B,X,length(X),length(Y))$可打印出$X$与$Y$的$LCS$

算法复杂性

- 时间复杂性
  - 计算代价的时间
    - （i,j）两层循环，i循环m步，j循环n步
    - $O(mn)$
  - 初始化的时间：$O(m+n)$
  - 总时间复杂性为：$O(mn)$
- 空间复杂性
  - 使用数组C和B
  - 需要空间$O(mn)$

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0										
2	p	0										
3	a	0										
4	n	0										
5	k	0										
6	i	0										
7	n	0										
8	g	0										

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0										
3	a	0										
4	n	0										
5	k	0										
6	i	0										
7	n	0										
8	g	0										

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↖	1←	1←	1←	1←	1←	1←	1←
3	a	0										
4	n	0										
5	k	0										
6	i	0										
7	n	0										
8	g	0										

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↖	1←	1←	1←	1←	1←	1←	1←
3	a	0	1↖	1←	1↑	1↑	1↑	2↖	2←	2←	2←	2←
4	n	0										
5	k	0										
6	i	0										
7	n	0										
8	g	0										

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↖	1←	1←	1←	1←	1←	1←	1←
3	a	0	1↖	1←	1↑	1↑	1↑	2↖	2←	2←	2←	2←
4	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↖
5	k	0										
6	i	0										
7	n	0										
8	g	0										

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↖	1←	1←	1←	1←	1←	1←	1←
3	a	0	1↖	1←	1↑	1↑	1↑	2↖	2←	2←	2←	2←
4	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↖
5	k	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↑
6	i	0										
7	n	0										
8	g	0										

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↖	1←	1←	1←	1←	1←	1←	1←
3	a	0	1↖	1←	1↑	1↑	1↑	2↖	2←	2←	2←	2←
4	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↖
5	k	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↑
6	i	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↖	3←	3↑
7	n	0										
8	g	0										

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↖	1←	1←	1←	1←	1←	1←	1←
3	a	0	1↖	1←	1↑	1↑	1↑	2↖	2←	2←	2←	2←
4	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↖
5	k	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↑
6	i	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↖	3←	3↑
7	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↑	3↑	4↖
8	g	0										

示例

i	j	0	1	2	3	4	5	6	7	8	9	10
		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑	0↑
2	p	0	0↑	0↑	1↖	1←	1←	1←	1←	1←	1←	1←
3	a	0	1↖	1←	1↑	1↑	1↑	2↖	2←	2←	2←	2←
4	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↖
5	k	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	2↑	2↑	3↑
6	i	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↖	3←	3↑
7	n	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↑	3↑	4↖
8	g	0	1↑	1↑	1↑	1↑	1↑	2↑	2↑	3↑	3↑	4↑

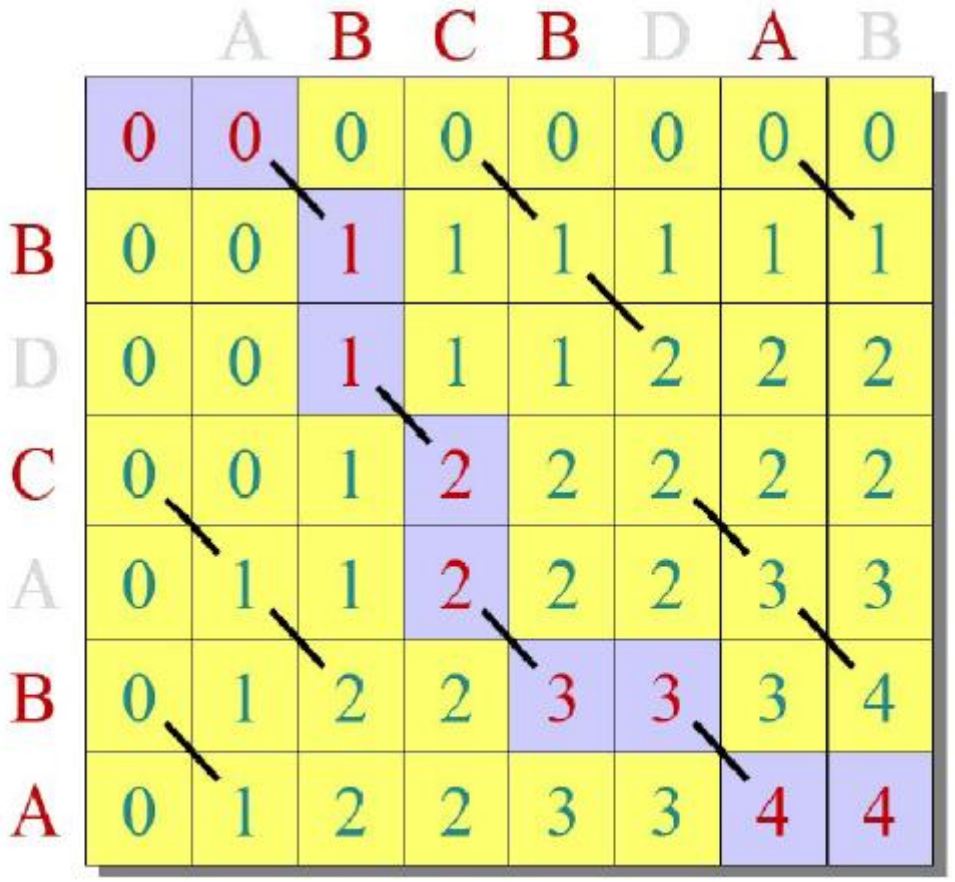
示例

	j	0	1	2	3	4	5	6	7	8	9	10
i		y_j	a	m	p	u	t	a	t	i	o	n
0	x_i		0	0	0	0	0	0	0	0	0	0
1	s	0	0	0	0	0	0	0	0	0	0	0
2	p	0	0	0	1	1	1	1	1	1	1	1
3	a	0	1	1	1	1	1	2	2	2	2	2
4	n	0	1	1	1	1	1	2	2	2	2	3
5	k	0	1	1	1	1	1	2	2	2	2	3
6	i	0	1	1	1	1	1	2	2	3	3	3
7	n	0	1	1	1	1	1	2	2	3	3	4
8	g	0	1	1	1	1	1	2	2	3	3	4
					p			a		i		n

最长公共子序列是***pain***

示例 2

$X =< A, B, C, B, D, A, B >; Y =< B, D, C, A, B, A >$



		A	B	C	B	D	A	B
	0	0	0	0	0	0	0	0
B	0	0	1	1	1	1	1	1
D	0	0	1	1	1	2	2	2
C	0	0	1	2	2	2	2	2
A	0	1	1	2	2	2	3	3
B	0	1	2	2	3	3	3	4
A	0	1	2	2	3	3	4	4

本讲内容

4.1 动态规划的原理

4.2 矩阵乘法问题

4.3 最长公共子序列

4.4 0-1背包问题

4.5 最优二叉搜索树

背包问题定义

给定n种物品和一个背包，物品i的重量是w_i，价值v_i，背包容量为C，问如何选择装入背包的物品，使装入背包中的物品的总价值最大？

根据是否对每类物品的数量进行限定，可分为两类问题：

- 完全背包问题（Unbounded Knapsack）：

  假设所有物品我们都有无限份

- 0-1背包问题（0-1 Knapsack）：

  假设所有物品，我们都至多一份，即每种物品只能选择完全装入或不装入，一个物品至多装入一次。



Capacity: 10

Item:	🐢	💡	🍉	🌮	🚒
Weight:	6	2	4	3	11
Value:	20	8	14	13	35

- 完全背包问题（Unbounded Knapsack）：
  - 假设所有物品，我们都有无限份
  - 问如何选择装入背包的物品，使装入背包中的物品的总价值最大？

🌮 🌮 💡 💡 Total weight: 10
Total value: 42

- 0-1背包问题（0-1 Knapsack）：
  - 假设所有物品，我们都只有一份
  - 问如何选择装入背包的物品，使装入背包中的物品的总价值最大？

💡 🍉 🌮 Total weight: 9
Total value: 35

符号定义

Item:					
Weight:	w_1	w_2	w_3	$\cdots$	w_n
Value:	v_1	v_2	v_3	$\cdots$	v_n



Capacity: C

完全背包问题

- 输入：$C > 0, w_i > 0, v_i > 0, 1 \le i \le n$
- 输出：$(h_1, h_2, \cdots, h_n)$，满足

$$\sum_{1 \le i \le n} w_i h_i \le C, \sum_{1 \le i \le n} v_i h_i \text{ 最大}$$

等价的整数规划问题

$$\max_{h} \sum_{1 \le i \le n} v_i h_i$$

$$s.t. \sum_{1 \le i \le n} w_i h_i \le C$$

0-1背包问题

- 输入：$C>0, w_i>0, v_i>0, 1\le i\le n$
- 输出：$(h_1, h_2, \cdots, h_n)$，$h_i \in \{0,1\}$，满足

$$\sum_{1\le i\le n} w_i h_i \le C, \sum_{1\le i\le n} v_i h_i \text{ 最大}$$

等价的0-1整数规划问题

$$\max_h \sum_{1\le i\le n} v_i h_i$$

$$s.t. \begin{cases} \sum_{1\le i\le n} w_i h_i \le C \\ h_i \in \{0,1\}\ (1\le i\le n) \end{cases}$$

利用暴力搜索方法求解0-1背包问题，每件物品都存在装入和不装入两种情况，所以总的时间复杂度是$O(2^n)$。

使用动态规划

- 分析优化解的结构 
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

优化子结构分析: try 1

- 子问题:
  - 考虑更小的背包



首先解小背包



然后，解大背包



然后，解更大的背包

优化子结构分析: try 1

- 这种方式失效
  - 每类物品只有一份
  - 求解子问题时，我们需要知道哪些物品已经被其子问题用过，哪些没有被用过

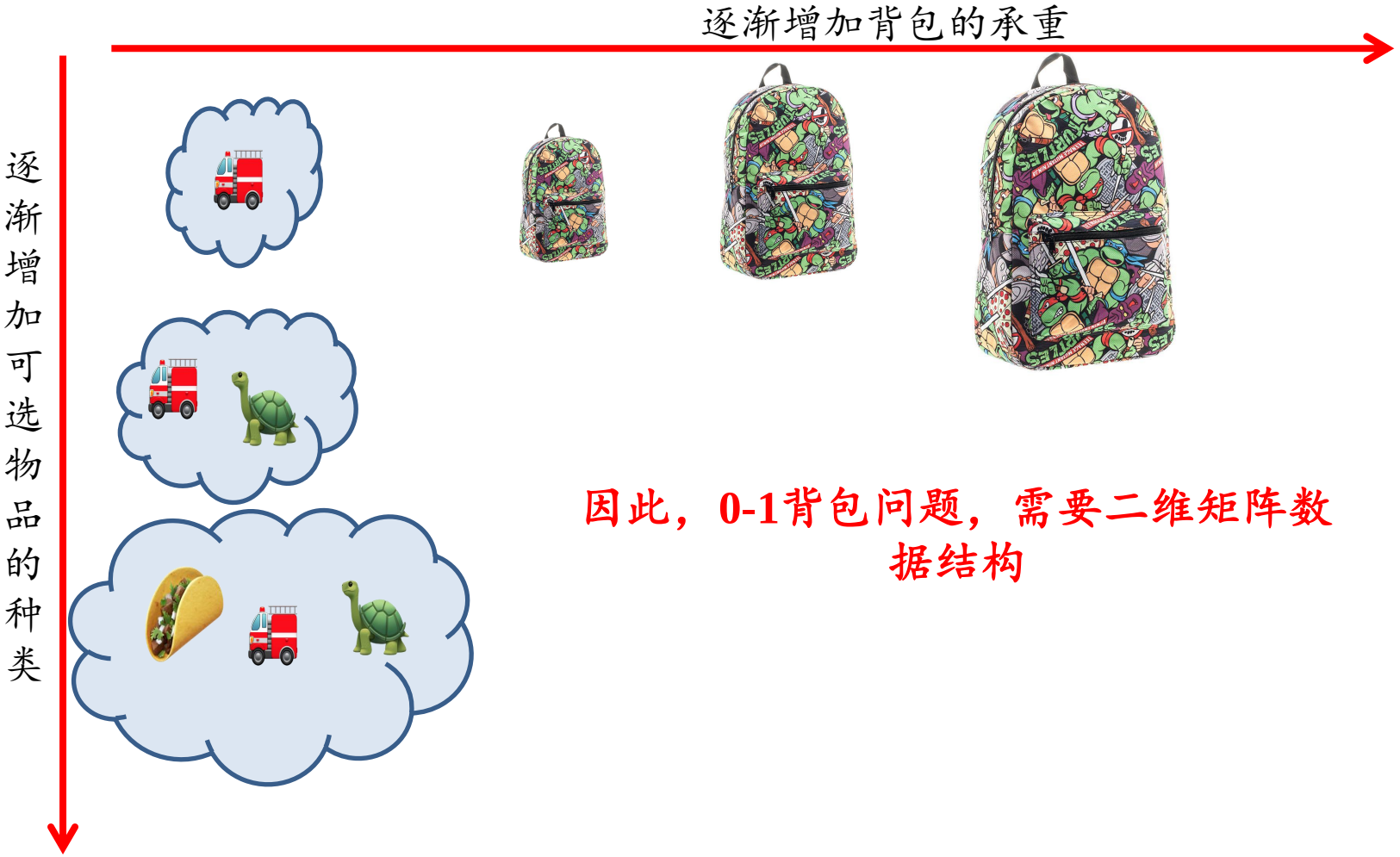






优化子结构分析: try 2

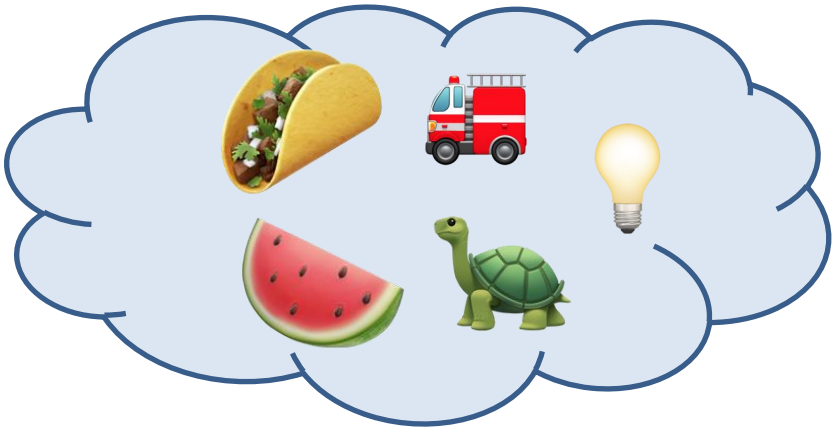
- 子问题:
  - 有更少物品种类的0-1 背包问题



因此，0-1背包问题，需要二维矩阵数据结构

优化子结构分析: try 2

- 二维矩阵数据结构包括x 和 j 两个维度



j ： 前j个物品



x：承重为x的背包

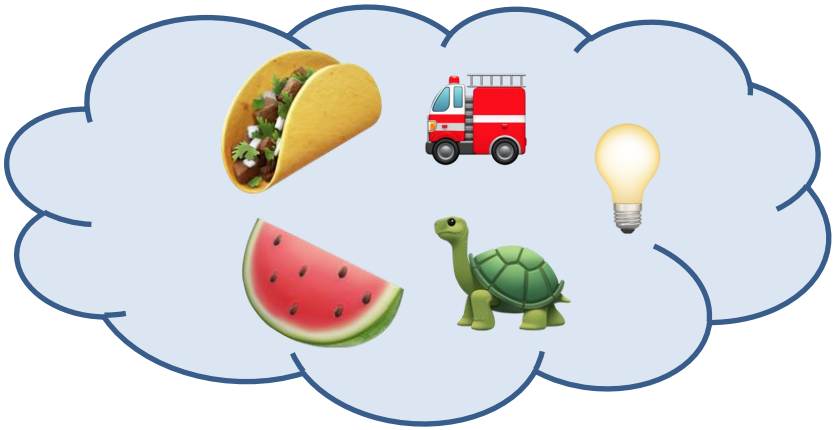
定义 $K[x, j]$ 为如下0-1背包问题的最优值

两种情况



item j

- **Case 1**: 前 j 个物品的子问题的最优解，不包含第 j 个物品
- **Case 2**: 前 j 个物品的子问题的最优解，包含了第 j 个物品



前 j 个物品



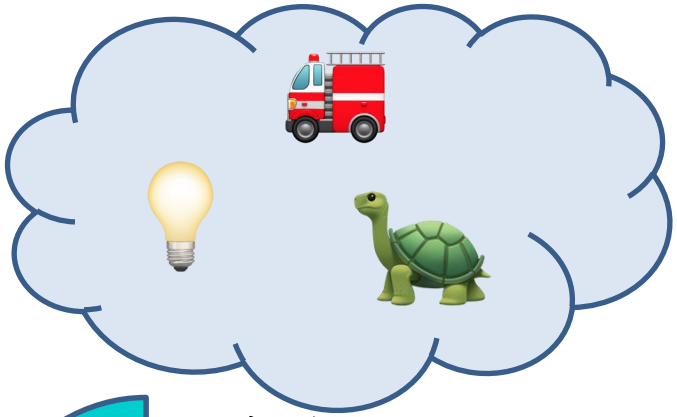
承重为 x 的背包

两种情况



item j

- **Case 1**: 前 j 个物品的子问题的最优解，不包含第 j 个物品



前 j 个物品





背包承重为 x，前 j 个物品的0-1背包问题，最大价值为 K

- 可以得到背包承重为 x，考虑前 $j-1$ 个物品的0-1背包问题



前 $j-1$ 个物品





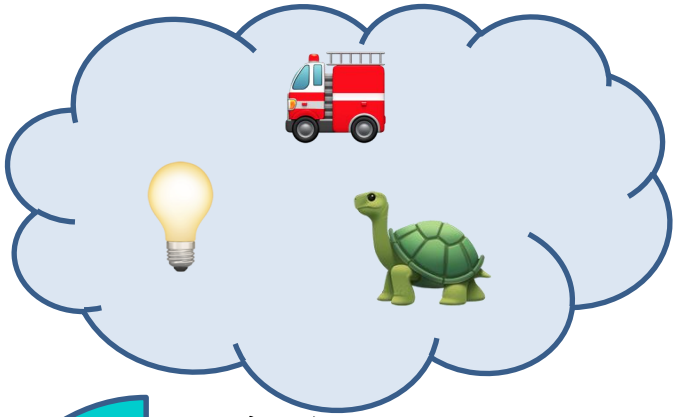
背包承重为 x，前 $j-1$ 个物品的0-1背包问题，最大价值为 $K[x,j-1]$

两种情况



item j

- **Case 2**: 前 j个物品的子问题的最优解，包含第j个物品



前j个物品





背包承重为x，前 j 个物品的0-1背包问题，最大价值为K

- 可以得到背包承重为$x - w_j$，考虑前 $j - 1$个物品的0-1背包问题



前$j-1$个物品





背包承重为$x - w_j$，前 $j - 1$个物品的0-1背包问题，最大价值为$K[x - w_j, j - 1] + v_j$

第一步：分析优化解的结构

定理（优化子结构）： 如果$(y_1, y_2, \cdots, y_n)$是0-1背包问题的优化解，则$(y_1, \cdots, y_{n-1})$是如下子问题的优化解：

$$\max_{h} \sum_{1 \le i \le n-1} v_i h_i$$

$$s.t. \begin{cases} \sum_{1 \le i \le n-1} w_i h_i \le C - w_n y_n \\ h_i \in \{0, 1\} \ (1 \le i \le n-1) \end{cases}$$

证明（反证法）：

如果$(y_1, \cdots, y_{n-1})$不是以上子问题优化解，则存在$(z_1, \cdots, z_{n-1})$是此子问题更优的解。于是，$(z_1, \cdots, z_{n-1}, y_n)$是原问题比$(y_1, y_2, \cdots, y_n)$更优的解，矛盾。

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价 
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

第二步：递归地定义最优值的代价

- 定义 $K[x, j]$ 为如下0-1背包问题的最优值：
  - 背包承重为 x，
  - 考虑前 j 个物品。

$$K[x, j] = max\{K[x, j-1], K[x - w_j, j-1] + v_j\}$$

Case 1　　Case 2

- 初始值
  - $K[x, 0] = 0$ 可选物品为空
  - $K[0, j] = 0$ 背包承重为0

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息 
- 根据构造最优值的信息构造优化解

第三步：自底向上地计算优化解的代价

Zero-One-Knapsack(W, weights, values)

Input: 背包承重C，每类物品对应的重量和价值数组weights, values

Output: 背包所能装的最大价值以及对应的所装物品

Zero-One-Knapsack(W,weights, values)

过程：Zero-One-Knapsack(W,weights, values)

```
1. n ← length(weights)
2. For   x ← 0   To   C   Do
3.     K[x,0] ← 0            /*初始化第0列*/
4. For   j ← 1   To   n    Do
5.     K[0,j] ← 0            /*初始化第0行*/
6. For   x ← 0   To   C   Do
7.     For   j ← 1   To   n    Do
8.         K[x,j] ← K[x,j-1]                    Case 1
9.         IF w_j ≤ x THEN
10.            K[x,j] ← max{K[x,j], K[x-w_j,j-1] + v_j}   Case 2
11.return K[C,n]
```

时间复杂度$O(nC)$

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解 

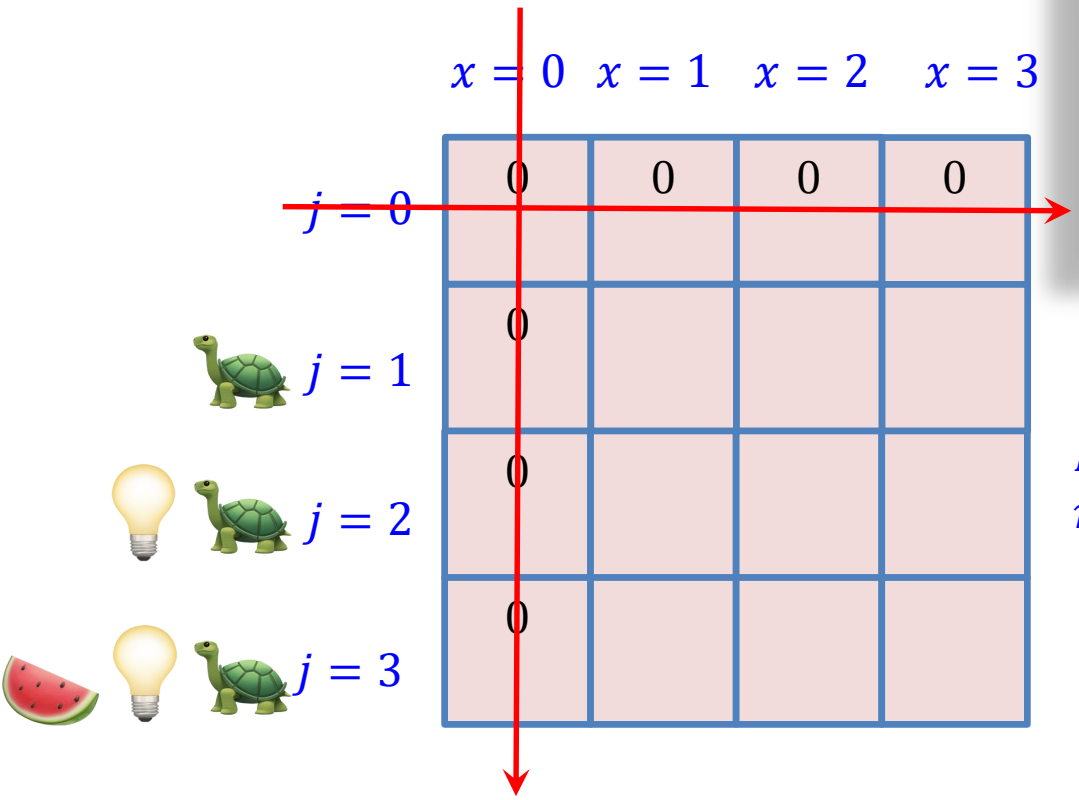
第四步：获取构造最优值的信息

过程：Zero-One-Knapsack(C,weights, values)

```
1. n ← length(weights)
2. For  x ← 0 To  C  Do
3.     For j ← 1 To n Do
4.         ITEMS[x,j] ← Ø
5. For   x ← 0   To   C   Do
6.     K[x,0] ← 0  /*初始化第0列*/
7. For   j ← 1   To   n    Do
8.     K[0,j] ← 0 /*初始化第0行*/
9. For   x ← 0   To   C   Do
10.    For   j ← 1   To   n    Do
11.        K[x,j] ← K[x,j-1]
12.        IF w_j ≤ x THEN
13.            K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.            IF k[x,j] was updated THEN
15.                ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.            ELSE
17.                ITEMS[x,j] ← ITEMS[x,j-1]
18.Return K[C,n], ITEMS[C,n]
```

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```



$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	0		
$j=2$	0			
$j=3$	0			

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

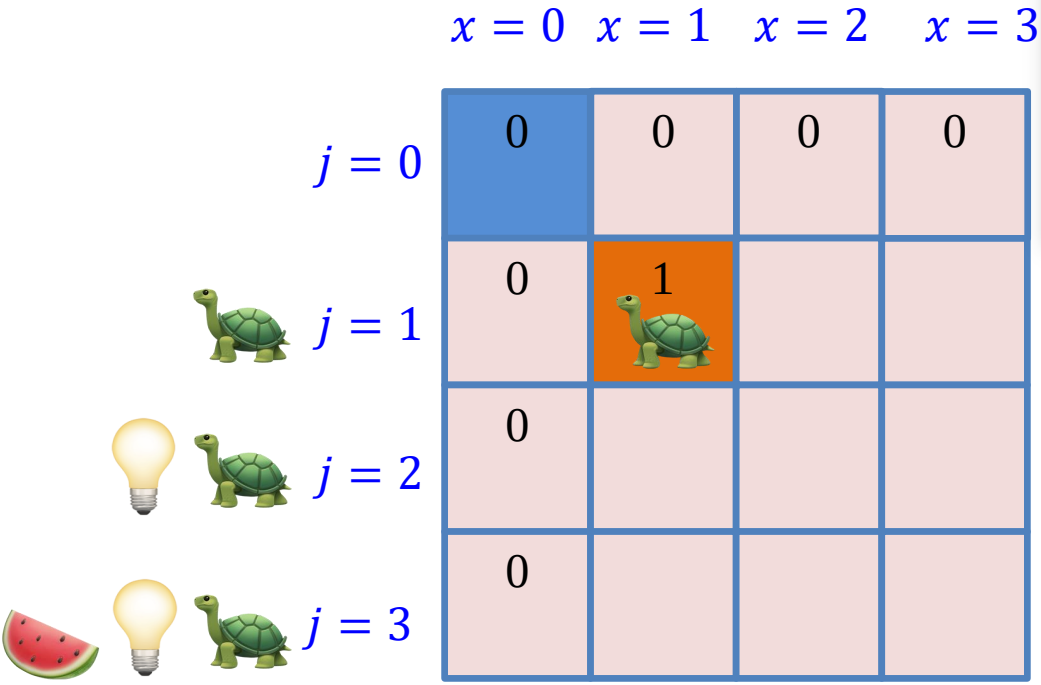
relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```



$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1		
$j=2$	0	1		
$j=3$	0			

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1		
$j=2$	0	1		
$j=3$	0	1		

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	0	
$j=2$	0	1		
$j=3$	0	1		

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	
$j=2$	0	1		
$j=3$	0	1		

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	
$j=2$	0	1	1	
$j=3$	0	1		

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	
$j=2$	0	1	4	
$j=3$	0	1		

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	
$j=2$	0	1	4	
$j=3$	0	1	4	

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	0
$j=2$	0	1	4	
$j=3$	0	1	4	

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	1
$j=2$	0	1	4	
$j=3$	0	1	4	

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	1
$j=2$	0	1	4	1
$j=3$	0	1	4	

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	1
$j=2$	0	1	4	5
$j=3$	0	1	4	

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	1
$j=2$	0	1	4	5
$j=3$	0	1	4	5

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	1
$j=2$	0	1	4	5
$j=3$	0	1	4	6

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

示例

```
1.    n ← length(weights)
2.    For  x ← 0 To  C  Do
3.        For j ← 1 To n Do
4.            ITEMS[x,j] ← ∅
5.    For   x ← 0   To   C   Do
6.        K[x,0] ← 0  /*初始化第0列*/
7.    For   j ← 1   To   n    Do
8.        K[0,j] ← 0 /*初始化第0行*/
9.    For   x ← 0   To   C   Do
10.       For   j ← 1   To   n    Do
11.           K[x,j] ← K[x,j-1]
12.           IF w_j ≤ x THEN
13.               K[x,j] ← max{ K[x,j], K[x - w_j,j-1] + v_j }
14.               IF k[x,j] was updated THEN
15.                   ITEMS[x,j] ← ITEMS[x - w_j,j-1] ∪ {ITEM j}
16.               ELSE
17.                   ITEMS[x,j] ← ITEMS[x,j-1]
18.   Return K[C,n], ITEMS[C,n]
```

	$x=0$	$x=1$	$x=2$	$x=3$
$j=0$	0	0	0	0
$j=1$	0	1	1	1
$j=2$	0	1	4	5
$j=3$	0	1	4	6

$$K[x,j] = max\{K[x,j-1], K[x-w_j,j-1] + v_j\}$$

current entry

relevant previous entry

Item:			
Weight:	1	2	3
Value:	1	4	6

Capacity: 3

本讲内容

4.1 动态规划的原理

4.2 矩阵乘法问题

4.3 最长公共子序列

4.4 0-1背包问题

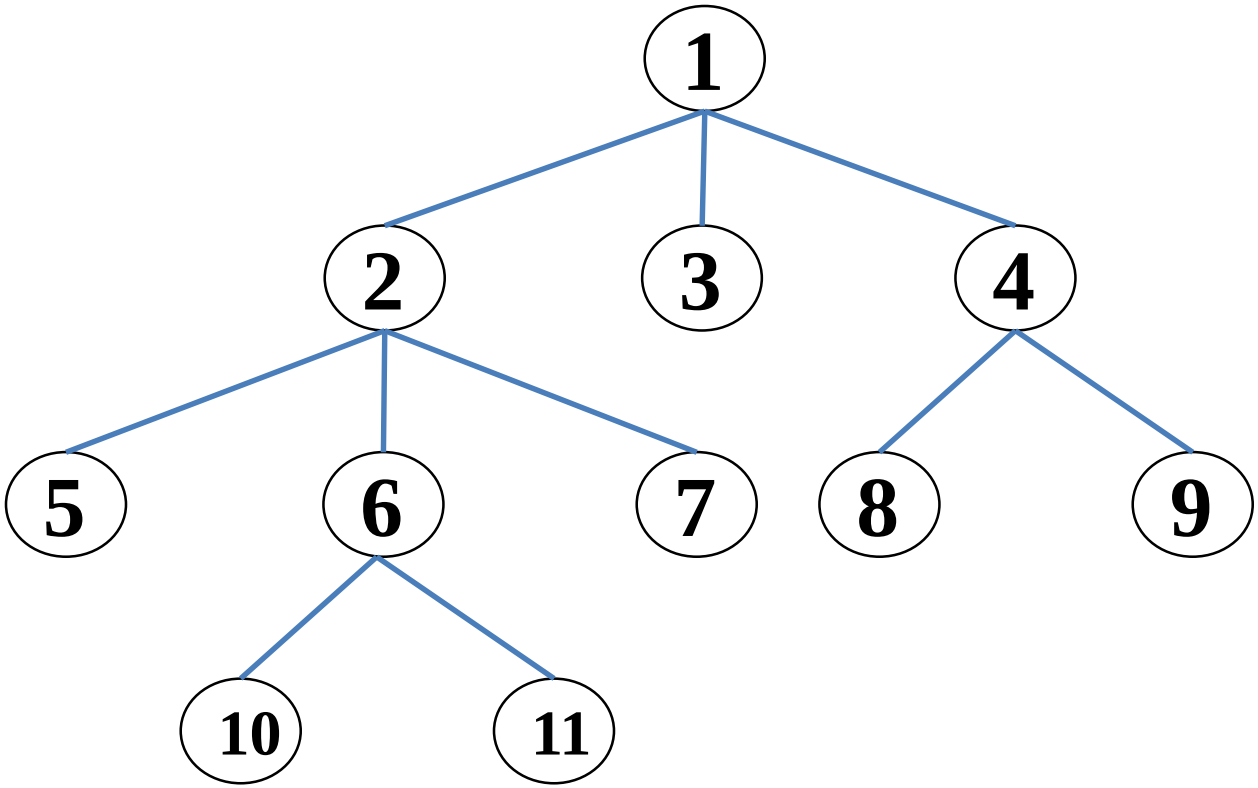
4.5 最优二叉搜索树

树

树形结构是一种重要的非线性结构。树是n个结点的有限集合，在任一棵非空树中：

（1）有且仅有一个称为根的结点。

（2）其余结点可分为m个互不相交的集合，而且这些集合中的每一集合都本身又是一棵树，称为根的子树。

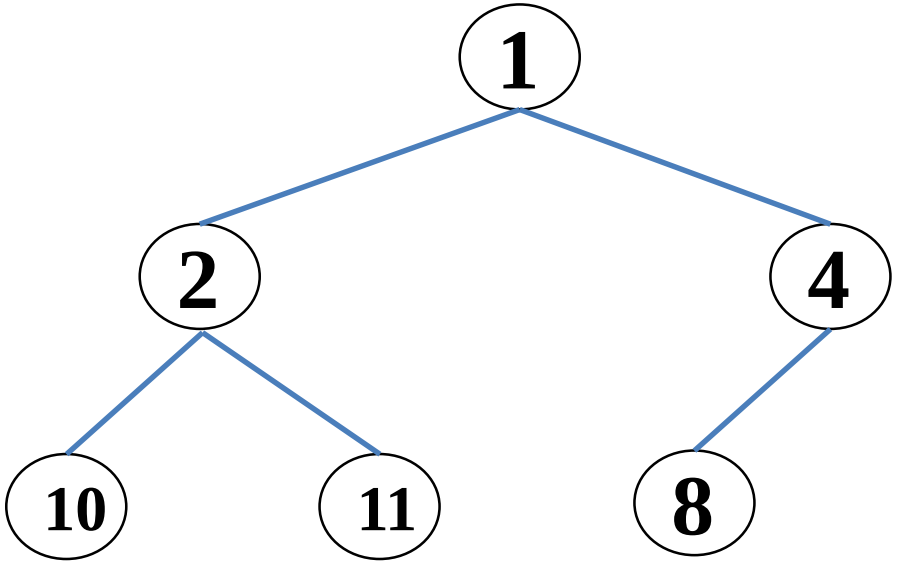


二叉树

概念：二叉树或为空树，或由根及两颗不相交的左、右子树构成，并且左、右子树本身也是二叉树。

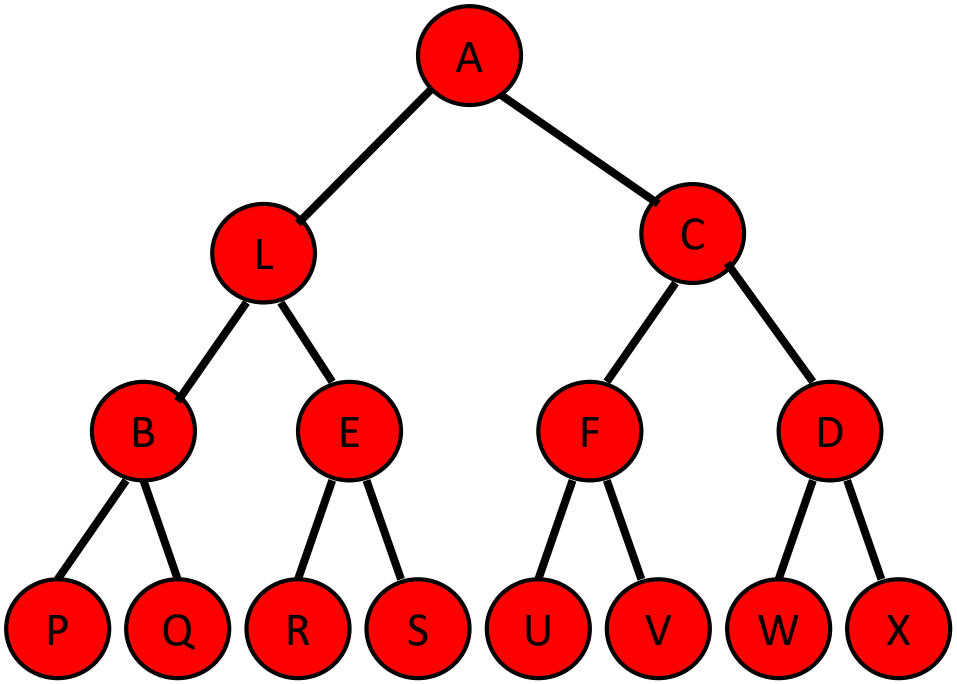
特点：

- 二叉树中每个结点最多有两棵子树；即二叉树每个结点度小于等于2；
- 左、右子树不能颠倒——有序树；
- 二叉树是递归结构，在二叉树的定义中又用到了二叉树的概念；



满二叉树

特殊二叉树，每层都是满的

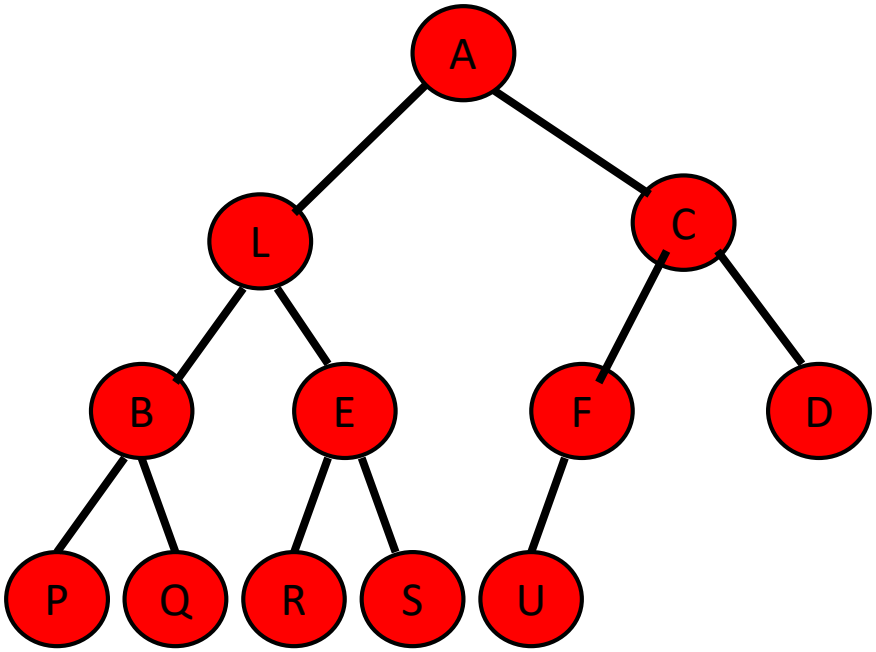


完全二叉树

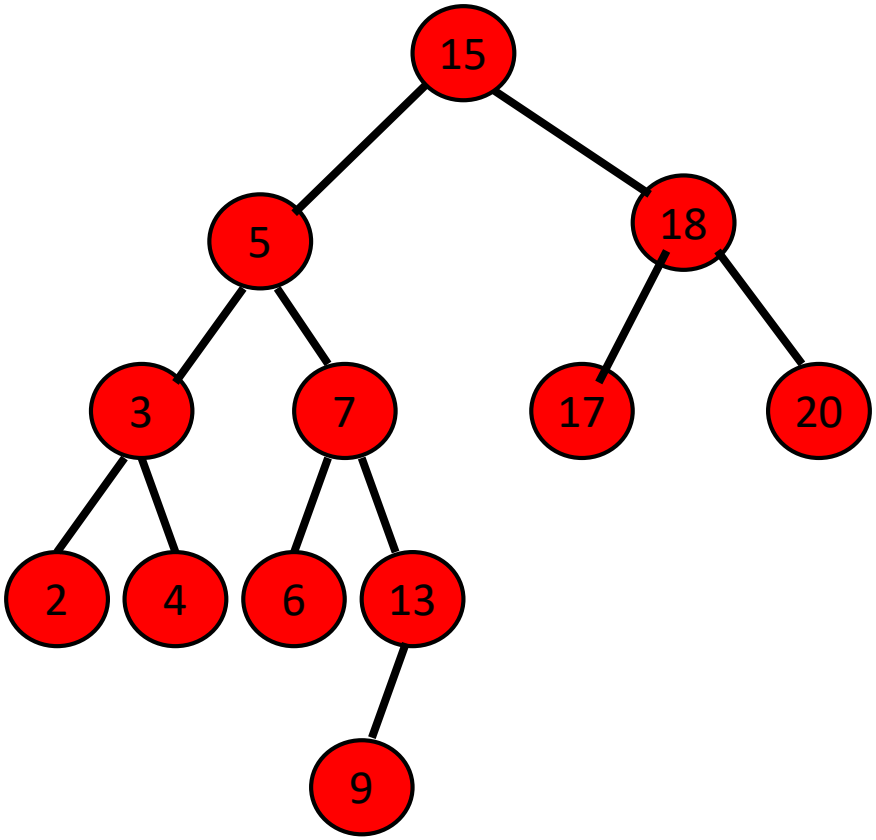
特殊二叉树，除了最后一层，每一层都是满的

满二叉树

从满二叉树最底层从右向左依次去掉若干结点形成的



二叉搜索树

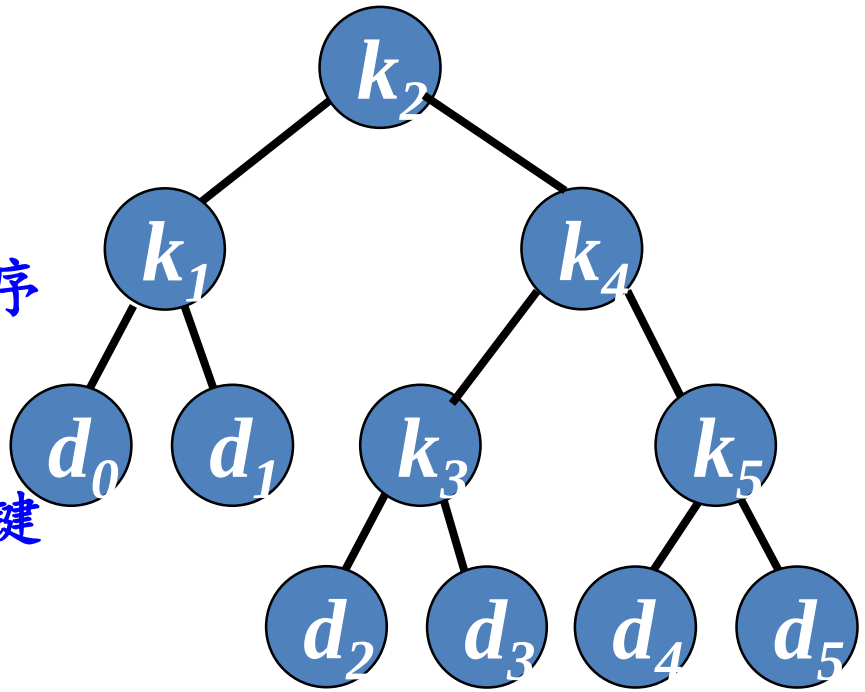


特殊二叉树：

- 左子树不空，则左子树所有结点值均小于该子树父结点值
- 右子树不空，则右子树所有结点值均大于该子树父结点值
- 左右子树都是二叉搜索树
- 中序遍历二叉查找树就可以得到一个有序序列

最优二叉搜索树 问题的定义

- 二叉搜索树T
  - 结点
    - $K=\{k_1,k_2,\cdots,k_n\}$：已排序的关键字序列
    - $D=\{d_0,d_1,\cdots,d_n\}$：伪关键字序列
    - d_i对应区间(k_i,k_{i+1})
    - d_0对应区间$(-\infty,k_1)$
    - d_n对应区间$(k_n,+\infty)$



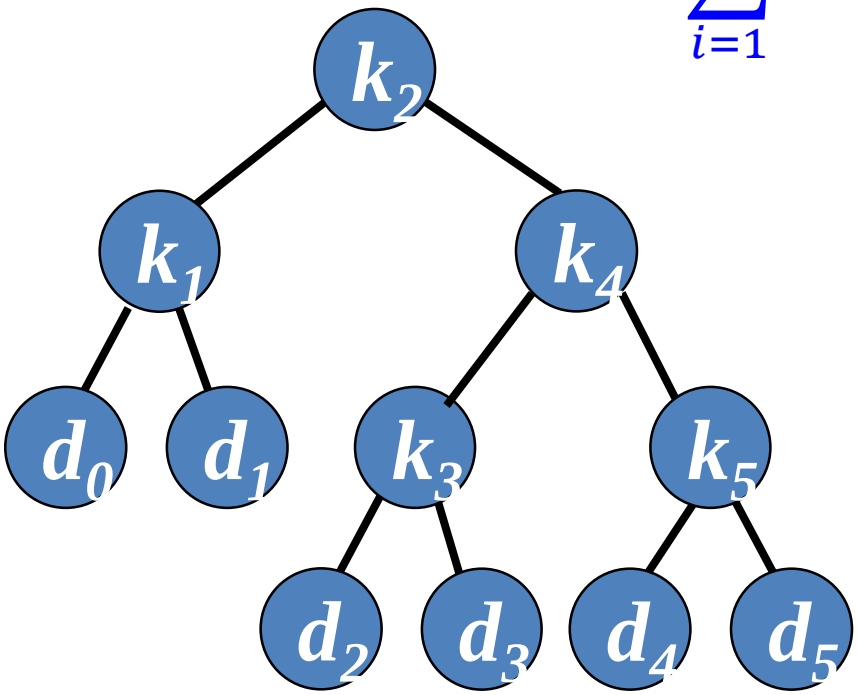
特点：对于树中结点X，左子树中所有关键字值小于X，右子树中所有关键字值大于X。

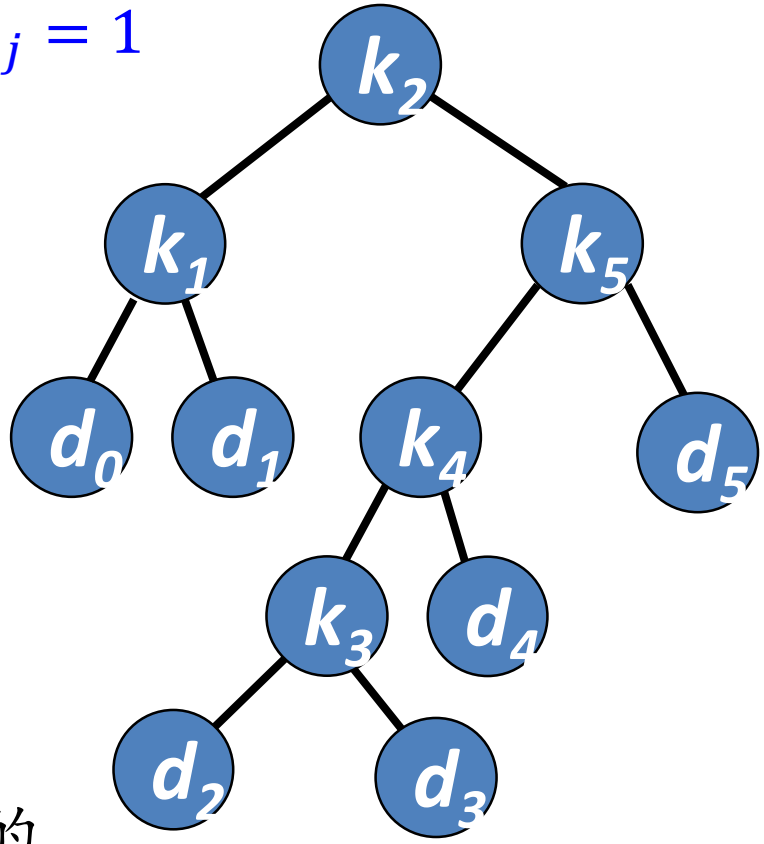
最优二叉搜索树 问题的定义

- 搜索k_i的概率为p_i
- 搜索d_i的概率为q_i

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$\sum_{i=1}^{n} p_i + \sum_{j=0}^{n} q_j = 1$$





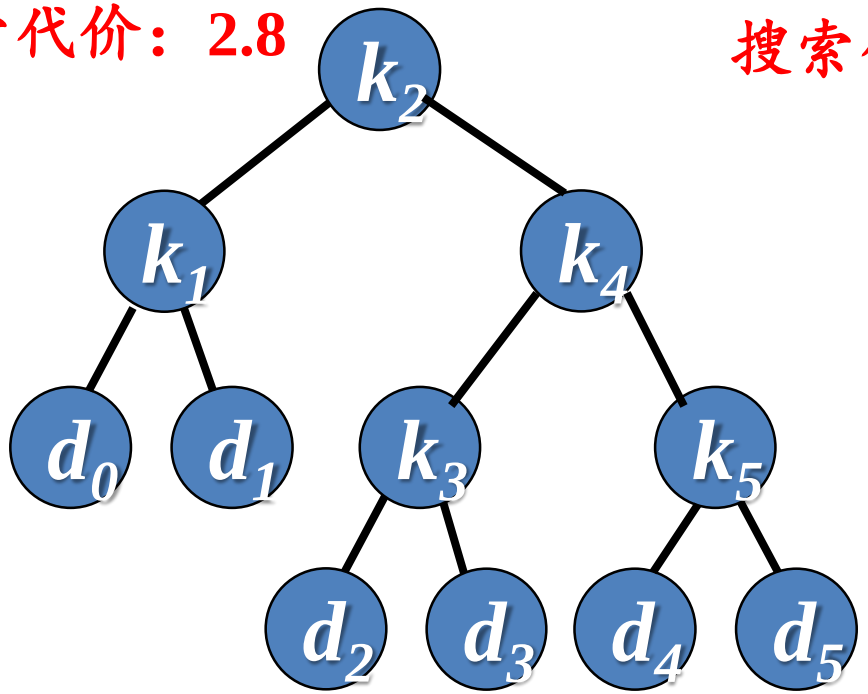
每个关键字的搜索代价**等于**需要访问的结点个数 **等于**关键字所在结点深度+1

最优二叉搜索树 问题的定义

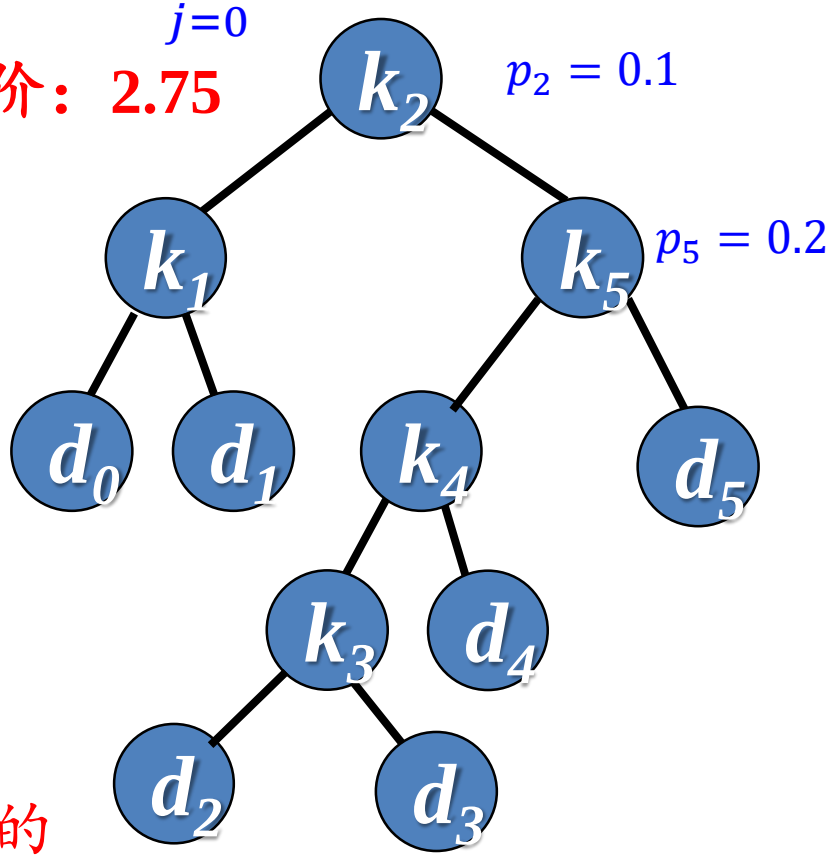
- 搜索树的期望代价

$$E(T) = \sum_{i=1}^{n} (\text{depth}_T(k_i) + 1)p_i + \sum_{j=0}^{n} (\text{depth}_T(d_i) + 1)q_i$$

搜索代价：2.8



搜索代价：2.75



- 最优二叉搜索树不一定是高度最矮的
- 概率高的关键字不一定出现在二叉搜索树的根结点

最优二叉搜索树 问题的定义

输入：

$K = \{k_1, k_2, \cdots, k_n\}$, $k_1 < k_2 < \cdots < k_n$, （有序的序列）

$D = \{d_0, d_1, \cdots, d_n\}$

$P = \{p_1, p_2, \cdots, p_n\}$, p_i为搜索关键字k_i的概率

$Q = \{q_0, q_1, \cdots, q_n\}$, q_i为搜索伪关键字d_i的概率

输出：

K的最优二叉搜索树T， $E(T)$最小

使用动态规划

- 分析优化解的结构 
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

优化子结构的分析

定理（优化子结构）：

如果最优二叉搜索树T具有包含关键字集合$\{k_i, k_{i+1}, \cdots, k_j\}$子树$T'$，则$T'$必是关于关键字集合$\{k_i, k_{i+1}, \cdots, k_j\}$子问题的优化解。

证明（反证法）：

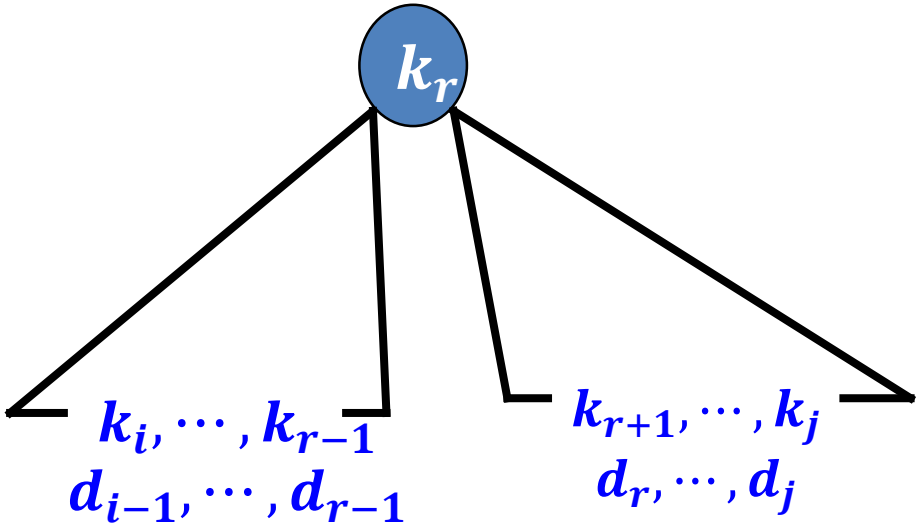
若不然，必有关键字集$\{k_i, k_{i+1}, \cdots, k_j\}$子树$T''$，$T''$的期望搜索代价低于$T'$；

用T''替换T中的T'，可以得到一个期望搜索代价比T小的原始问题的二叉搜索树，与T是最优解矛盾。

为什么是$\{k_i, k_{i+1}, \cdots, k_j\}$？

优化子结构的分析

- 用优化子结构从子问题的优化解构造原问题的优化解
  - $K = \{k_i, k_{i+1}, \cdots, k_j\}$的优化解的根结点为$K$中某个$k_r$；
  - 对于$k_r \in K$，确定$\{k_i, \cdots, k_{r-1}\}$和$\{k_{r+1}, \cdots, k_j\}$的优化解，我们就可以求出K的优化解。



空子树：

如果$r = i$, 左子树$\{k_i, \cdots, k_{r-1}\}$不包含任何关键字，仅包含伪关键字d_{i-1}

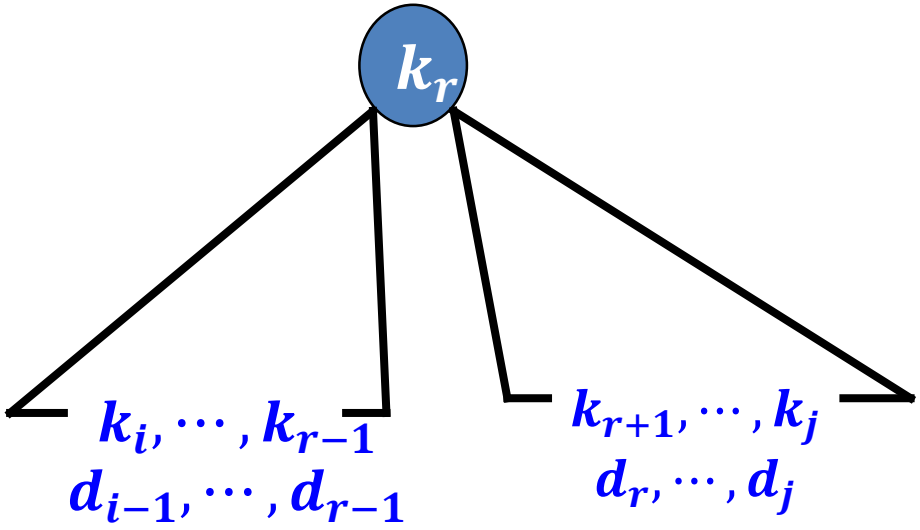
如果$r = j$, 右子树$\{k_{r+1}, \cdots, k_j\}$不包含任何关键字，仅包含伪关键字d_j

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价 
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解

建立优化解的搜索代价递归方程

- 令$E(i,j)$为$\{k_i,\cdots,k_j\}$和$\{d_{i-1},d_i,\cdots,d_j\}$的优化解$T_{ij}$的期望搜索代价，$W(i,j)$为$\{k_i,\cdots,k_j\}$和$\{d_{i-1},\cdots,d_j\}$的概率之和
  - 当$j=i-1$时，T_{ij}中只有叶结点d_{i-1}，$E(i,i-1)=q_{i-1}$
  - 当$j\geq i$时，选择一个$K_r\in\{k_i,\cdots,k_j\}$:

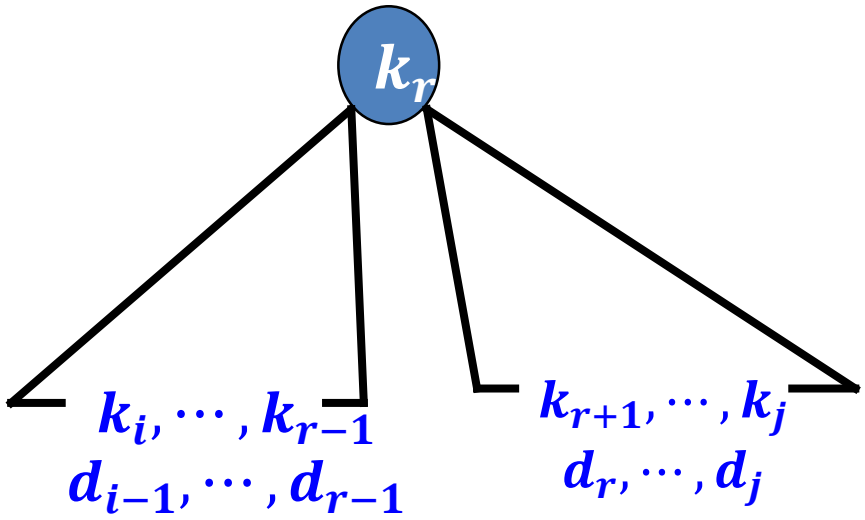


$$E(i,j)=P_r+(E(i,r-1)+W(i,r-1))+(E(r+1,j)+W(r+1,j))$$

左子树　　右子树

建立优化解的搜索代价递归方程

- 当把左右优化子树放进T_{ij}时，每个结点的深度增加1



1、$W(i,r-1)=\sum_{l=i}^{r-1}P_l+\sum_{l=i-1}^{r-1}q_l$

2、$W(r+1,j)=\sum_{l=r+1}^{j}P_l+\sum_{l=r}^{j}q_l$

3、$W(i,j)=W(i,r-1)+W(r+1,j)+P_r$

$=\sum_{l=i}^{r-1}P_l+\sum_{l=r+1}^{j}P_l+\sum_{l=i-1}^{r-1}q_l+\sum_{l=r}^{j}q_l+P_r$

$=\sum_{l=i}^{j}P_l+\sum_{l=i-1}^{j}q_l$

因此，$W(i,j)$与r没有关系，无论在哪里分，$W(i,j)$都是相同的！

$$E(i,j)=E(i,r-1)+E(r+1,j)+W(i,j)$$

建立优化解的搜索代价递归方程

$$E(i,j)=\begin{cases} q_{i-1}=q_j & if\ \ j=i-1 \\ \min\limits_{i\le r\le j}\{E(i,r-1)+E(r+1,j)+W(i,j)\} & if\ j\ge i \end{cases}$$

$$W(i,j)=\sum_{l=i}^{j}P_l+\sum_{l=i-1}^{j}q_l$$

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息 
- 根据构造最优值的信息构造优化解

自下而上计算优化解的搜索代价

$q_0 =$ $E(1,0)$ $E(1,1)$ $E(1,2)$ $E(1,3)$ $E(1,4)$

$q_1 =$ $E(2,1)$ $E(2,2)$ $E(2,3)$ $E(2,4)$

$q_2 =$ $E(3,2)$ $E(3,3)$ $E(3,4)$

$q_3 =$ $E(4,3)$ $E(4,4)$

$q_4 =$ $E(5,4)$

矩阵E有什么特点？

行号i从1到$n+1$

列号j从0到n（n是关键字的个数）

$$E(i,j) = \begin{cases} q_{i-1} = q_j & if\ \ j = i-1 \\ \min\limits_{i \le r \le j} \{E(i,r-1) + E(r+1,j) + W(i,j)\} & if\ j \ge i \end{cases}$$

自下而上计算优化解的搜索代价

$W(i,j)$与r没有关系，无论在哪里分，$W(i,j)$都是相同的！设定从j处分

$$W(i,i-1)=q_{i-1};\quad W(j+1,j)=q_j$$

$$W(i,j)=W(i,j-1)+W(j+1,j)+p_j$$

$$=W(i,j-1)+p_j+q_j$$

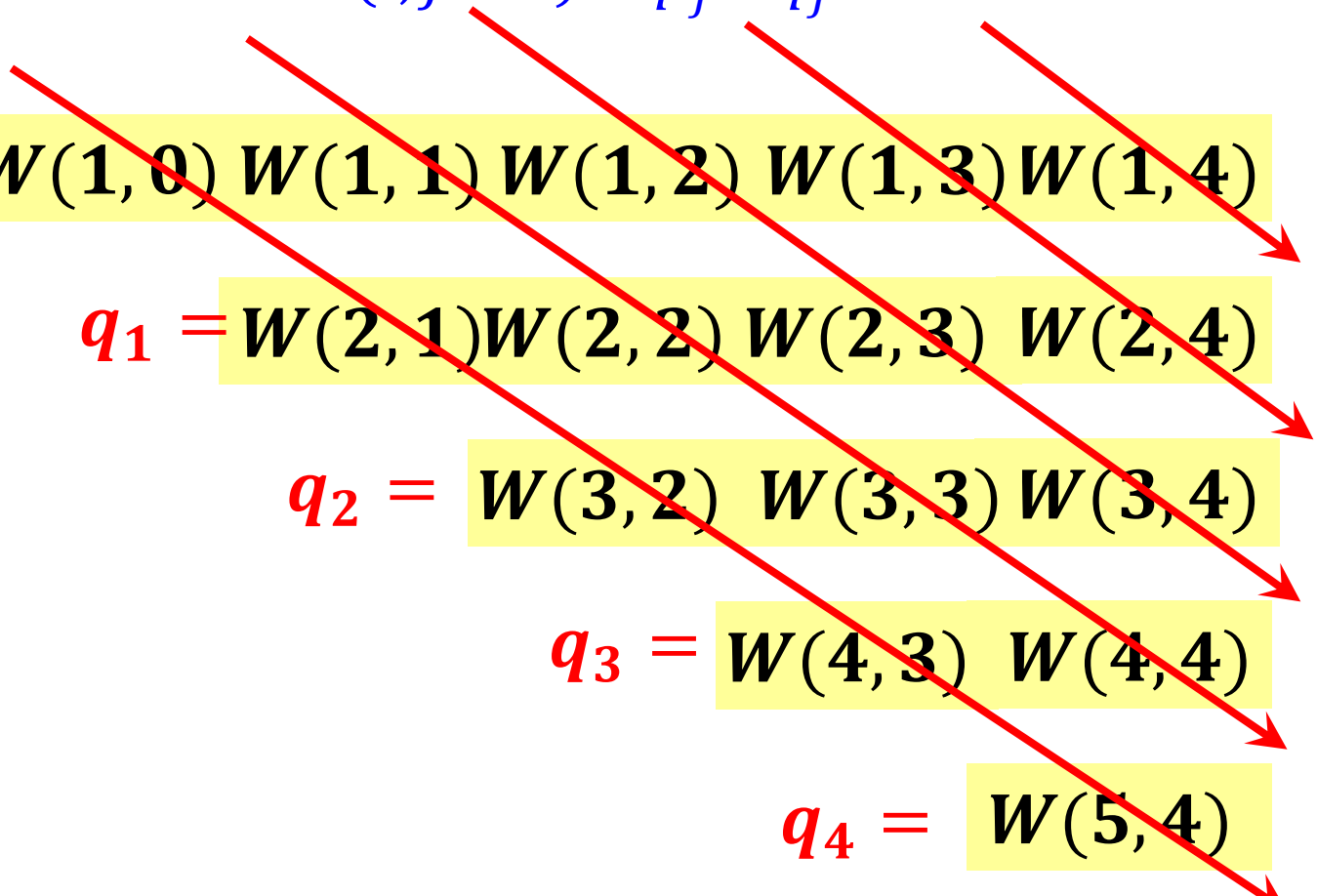
$$q_0 = W(1,0)\ W(1,1)\ W(1,2)\ W(1,3)\ W(1,4)$$

$$q_1 = W(2,1)\ W(2,2)\ W(2,3)\ W(2,4)$$

$$q_2 = W(3,2)\ W(3,3)\ W(3,4)$$

$$q_3 = W(4,3)\ W(4,4)$$

$$q_4 = W(5,4)$$



自下而上计算优化解的搜索代价

- 数据结构
  - $E[1:n+1;\ 0:n]$: 存储优化解搜索代价
  - $W[1:\ n+1;\ 0:n]$: 存储代价增量
  - $Root[1:n;\ 1:n]$: $root(i,j)$记录子问题$\{k_i,\cdots,k_j\}$优化解的根结点

自下而上计算优化解的搜索代价

```
Optimal-BST
Input:    K和D的概率数组p、q
Output:   搜索代价E，根结点Root
执行: Optimal-BST(p, q)
过程: Optimal-BST(p, q)
1. n ← length(p)
2. For  i ← 1  To  n+1   Do
3.     E(i, i-1) ← q_{i-1};
4.     W(i, i-1) ← q_{i-1};
5. For  l ← 1  To  n   Do
6.     For   i ← 1   To   n-l+1   Do
7.         j ← i+l-1;
8.         E(i, j) ← ∞;
9.         W(i, j) ← W(i, j-1)+p_j+q_j;
10.        For  r=i  To  j   Do
11.            t ← E(i, r-1) + E(r+1, j) + W(i, j);
12.            IF    t<E(i, j) Then
13.                E(i, j) ← t;  Root(i, j) ← r;
14.Return  E and  Root
```

使用动态规划

- 分析优化解的结构
- 递归地定义最优值的代价
- 自底向上地计算优化解的代价保存之，并获取构造最优值的信息
- 根据构造最优值的信息构造优化解 

思考：优化解的构造算法

示例

$$E(i,j)=\begin{cases} q_{i-1}=q_j & if\ \ j=i-1 \\ \min\limits_{i\le r\le j}\{E(i,r-1)+E(r+1,j)+W(i,j)\} & if\ j\ge i \end{cases}$$

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$E(1,3)=min\begin{cases} E(1,0)+E(2,3)+W(1,3) \\ =0.05+0.70+0.55=1.30 \quad r=1 \\ E(1,1)+E(3,3)+W(1,3) \\ =0.45+0.25+0.55=1.25 \quad r=2 \\ E(1,2)+E(4,3)+W(1,3) \\ =0.90+0.05+0.55=1.50 \quad r=3 \end{cases}$$

0	1	2	3	4	5	
0.05	0.45	0.90	1.25	1.75	2.75	1
	0.10	0.40	0.70	1.20	2.00	2
		0.05	0.25	0.60	1.30	3
			0.05	0.30	0.90	4
				0.05	0.50	5
					0.10	6

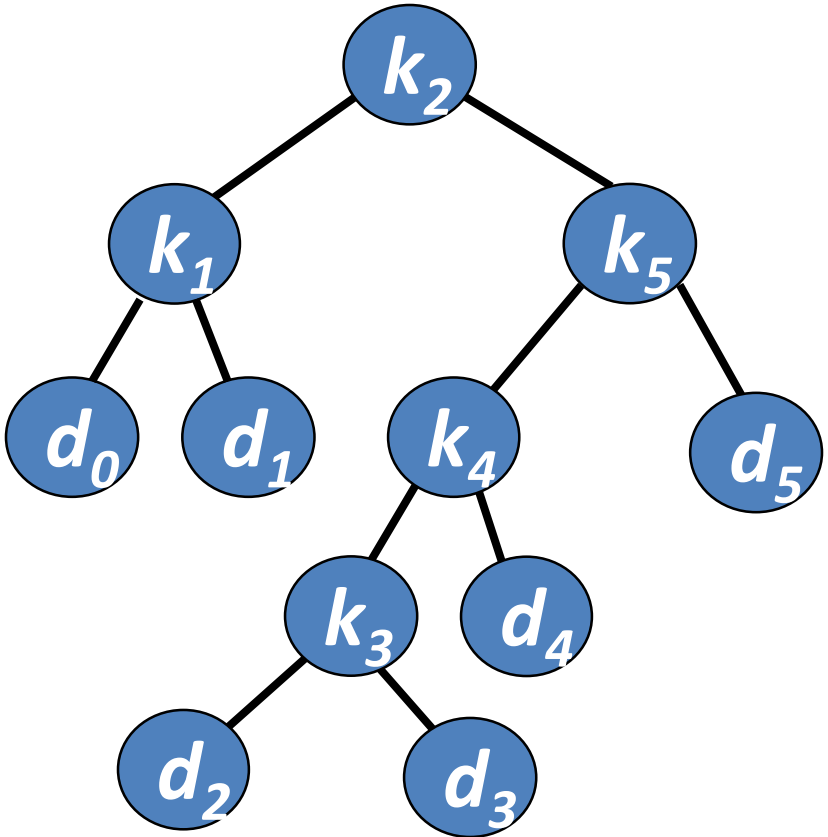
(a) $E(i,j)$

0	1	2	3	4	5
0.05	0.30	0.45	0.55	0.70	1.00
	0.10	0.25	0.35	0.50	0.80
		0.05	0.15	0.30	0.60
			0.05	0.20	0.50
				0.05	0.35
					0.10

(b) $W(i,j)$

1	2	3	4	5
1	1	2	2	2
	2	2	2	4
		3	4	5
			4	5
				5

(c) $Root(i,j)$



示例

$$E(i,j)=\begin{cases} q_{i-1}=q_j & if\ \ j=i-1 \\ \min\limits_{i\le r\le j}\{E(i,r-1)+E(r+1,j)+W(i,j)\} & if\ j\ge i \end{cases}$$

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

$$E(1,3)=min\begin{cases} E(1,0)+E(2,3)+W(1,3) \\ =0.05+0.70+0.55=1.30 \quad r=1 \\ E(1,1)+E(3,3)+W(1,3) \\ =0.45+0.25+0.55=1.25 \quad r=2 \\ E(1,2)+E(4,3)+W(1,3) \\ =0.90+0.05+0.55=1.50 \quad r=3 \end{cases}$$

0	1	2	3	4	5
0.05	0.30	0.45	0.55	0.70	1.00
	0.10	0.25	0.35	0.50	0.80
		0.05	0.15	0.30	0.60
			0.05	0.20	0.50
				0.05	0.35
					0.10

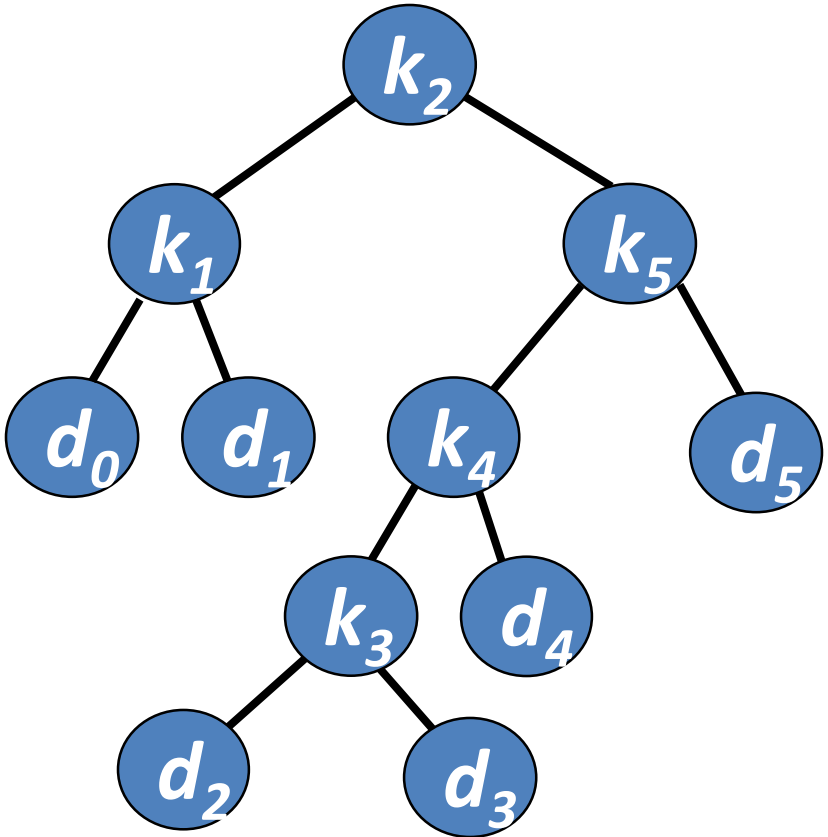
(a) $W(i,j)$

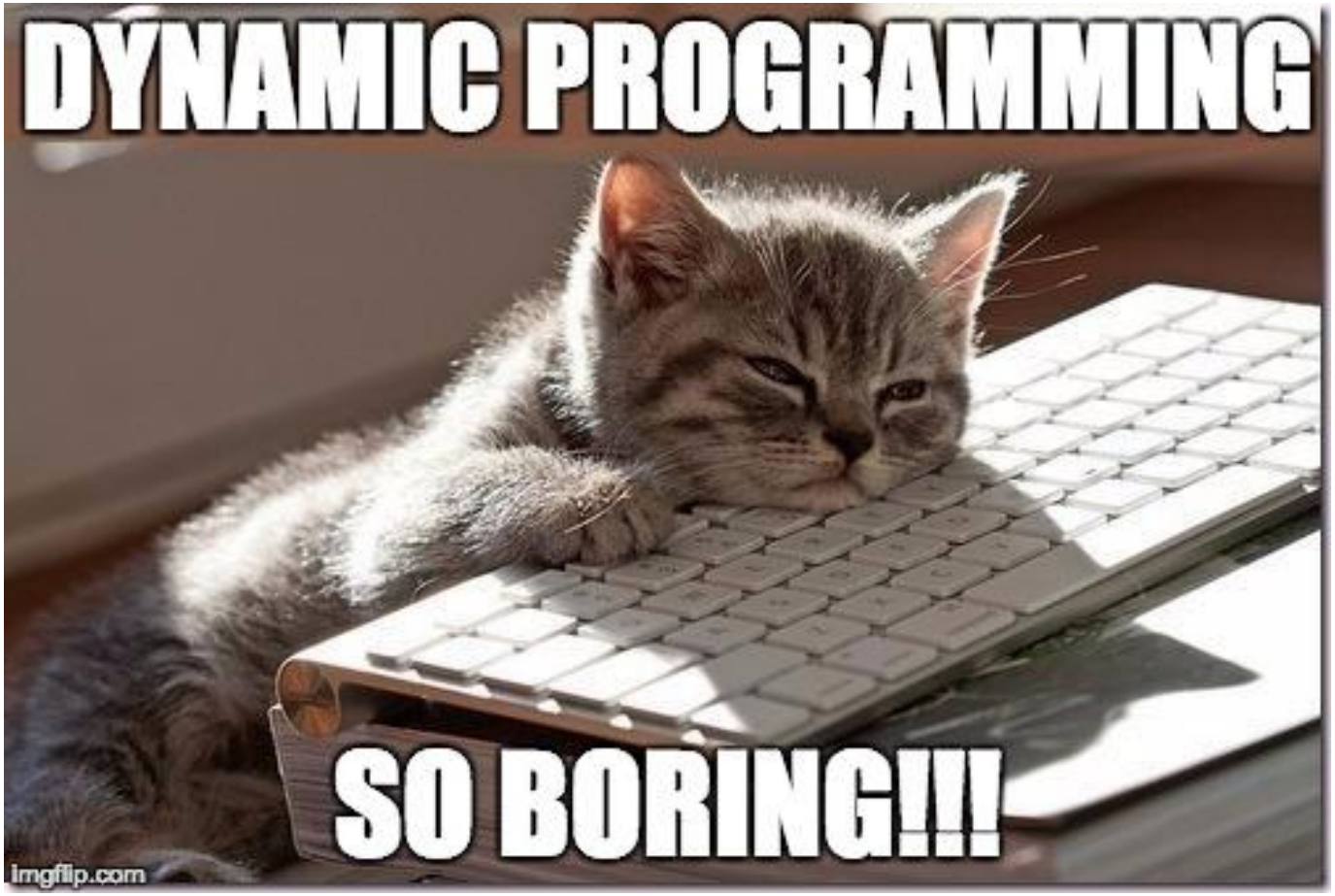
0	1	2	3	4	5	
0.05	0.45	0.90	1.25	1.75	2.75	1
	0.10	0.40	0.70	1.20	2.00	2
		0.05	0.25	0.60	1.30	3
			0.05	0.30	0.90	4
				0.05	0.50	5
					0.10	6

(b) $E(i,j)$

1	2	3	4	5
1	1	2	2	2
	2	2	2	4
		3	4	5
			4	5
				5

(c) $Root(i,j)$




DYNAMIC PROGRAMMING
SO BORING!!!
imgflip.com